



AccoTEST STS8300 应用培训

FXVle 单板硬件及应用

2022-1-11

Make Testing More Valuable

目录

- 单板特性
- 指标
- 原理框图及外观结构
- 脉冲输出功率曲线
- 槽位及对应PogoPin定义
- 原理图设计、PCB布局注意事项
- VI源功能
 - Kelvin接触电阻检测
 - Turbo功能
 - FANOUT
 - exFANOUT
- AWG及Digitizer使用样例
- TMU使用样例
- DVM使用样例

- 单板特性

SM8233 十二通道浮动电压电流源模块是 STS 8300 系统中的标准模块，模块提供十二路精密四象限 VI 源通道，每六个通道为一组，两组独立全浮动。每一组有两路时间测量通道，每一组内支持任意两通道差分测量功能。

• 单板特性

- 单板十二通道分组浮动，每组六通道（每组共用一路 FL 和 SL）
- 全四象限输出电压电流
- 带远端地线补偿的 Kelvin 连接
- 通断检测功能
- 3 个电压量程档：- $\pm 30V$ ， $\pm 10V$ ， $\pm 3.6V$
- 6 个电流量程档：- $\pm 1A$ (脉冲)， $\pm 100mA$ ， $\pm 10mA$ ， $\pm 1mA$ ， $\pm 100\mu A$ ， $\pm 10\mu A$
- 电流测量放大能力：- X1， X10
- 每通道电压电流可同时测量
- 每通道具备独立电压测量模式（Relay Sense ON Mode）
- 每通道具备硬件数据处理功能，包括最大值、最小值、平均值及跳变位置
- 每通道具备可编程窗口式电压及电流箝位
- 每通道具备独立的限流保护功能
- 每组两路时间测量单元(TMU)
- 时间测量单元测量的时间参数，包括频率/周期、占空比、脉宽、上升/下降时间、传输延时、事件计数等
- 时间测量单元有 3 个时间量程：30ns~950ns， 800ns~4ms， 30ns~5.368s，
- 时间测量单元有 3 个电压量程： $\pm 30V$ / $\pm 10V$ / $\pm 3.6V$
- 时间测量单元的滤波器为 PASS/1M/100K
- 支持每一组内任意两通道差分测量功能(DVM)，电压档位为 3.6V
- 每一组内有 2 个通道支持扇出功能
- 隔离电压 200V
- 支持与 STS 8300 系统内模块同步功能
- 支持多工位并行测试工作模式
- 本板保存校准数据
- 本板自带温湿度监测功能

- 指标

Voltage Force

Range	Resolution(16-Bit)	Accuracy
±30V	1.017mV	±(6mV + 0.025% Rdg)
±10V	0.339mV	±(2mV + 0.025% Rdg)
±3.6V	0.113mV	±(0.75mV + 0.025% Rdg)

Voltage Measure

Range	Resolution(16-Bit)	Accuracy
±30V	1.144mV	±(6mV + 0.025% Rdg)
±10V	0.381mV	±(2mV + 0.025% Rdg)
±3.6V	0.127mV	±(0.75mV + 0.025% Rdg)

• 指标

Current Force

Range	Resolution(16-Bit)	Accuracy
±1A	33.908uA	$\pm(0.2\text{mA} + 0.05\% \text{ Rdg} + 20\text{uA/V})$
±100mA	3.39uA	$\pm(15\text{uA} + 0.05\% \text{ Rdg} + 2\text{uA/V})$
±10mA	339.08nA	$\pm(1.5\text{uA} + 0.05\% \text{ Rdg} + 0.2\text{uA/V})$
±1mA	33.908nA	$\pm(150\text{nA} + 0.05\% \text{ Rdg} + 25\text{nA/V})$
±100uA	3.39nA	$\pm(25\text{nA} + 0.05\% \text{ Rdg} + 3\text{nA/V})$
±10uA	0.3815nA	$\pm(25\text{nA} + 0.05\% \text{ Rdg} + 3\text{nA/V})$

Current Measure(X1 amplifier)

Range	Resolution(16-Bit)	Accuracy
±1A	38.147uA	$\pm(0.2\text{mA} + 0.05\% \text{ Rdg} + 20\text{uA/V})$
±100mA	3.815uA	$\pm(15\text{uA} + 0.05\% \text{ Rdg} + 2\text{uA/V})$
±10mA	381.47nA	$\pm(1.5\text{uA} + 0.05\% \text{ Rdg} + 0.2\text{uA/V})$
±1mA	38.147nA	$\pm(150\text{nA} + 0.05\% \text{ Rdg} + 25\text{nA/V})$
±100uA	3.8147nA	$\pm(25\text{nA} + 0.05\% \text{ Rdg} + 3\text{nA/V})$
±10uA	0.3815nA	$\pm(25\text{nA} + 0.05\% \text{ Rdg} + 3\text{nA/V})$

Current Measure(X10 amplifier)

Range	Resolution(16-Bit)	Accuracy
±1A	3.815uA	$\pm(70\text{uA} + 0.05\% \text{ Rdg} + 6\text{uA/V})$
±100mA	381.47nA	$\pm(7\text{uA} + 0.05\% \text{ Rdg} + 0.6\text{uA/V})$
±10mA	38.147nA	$\pm(0.7\text{uA} + 0.05\% \text{ Rdg} + 60\text{nA/V})$
±1mA	3.8147nA	$\pm(70\text{nA} + 0.05\% \text{ Rdg} + 6\text{nA/V})$
±100uA	0.3815nA	$\pm(15\text{nA} + 0.05\% \text{ Rdg} + 0.8\text{nA/V})$
±10uA	0.0381nA	$\pm(5\text{nA} + 0.05\% \text{ Rdg} + 0.1\text{nA/V})$

• 指标

Clamp Voltage Ranges	Resolution	Accuracy (%FS)
±30V, ±10V, ±3.6V	16bit	±0.5%
Clamp Current Ranges	Resolution	Accuracy (%FS)
±100mA, ±10mA, ±1mA, ±100uA, ±10uA	16bit	±0.5%
±1A(Pulse)	16bit	±1.6%

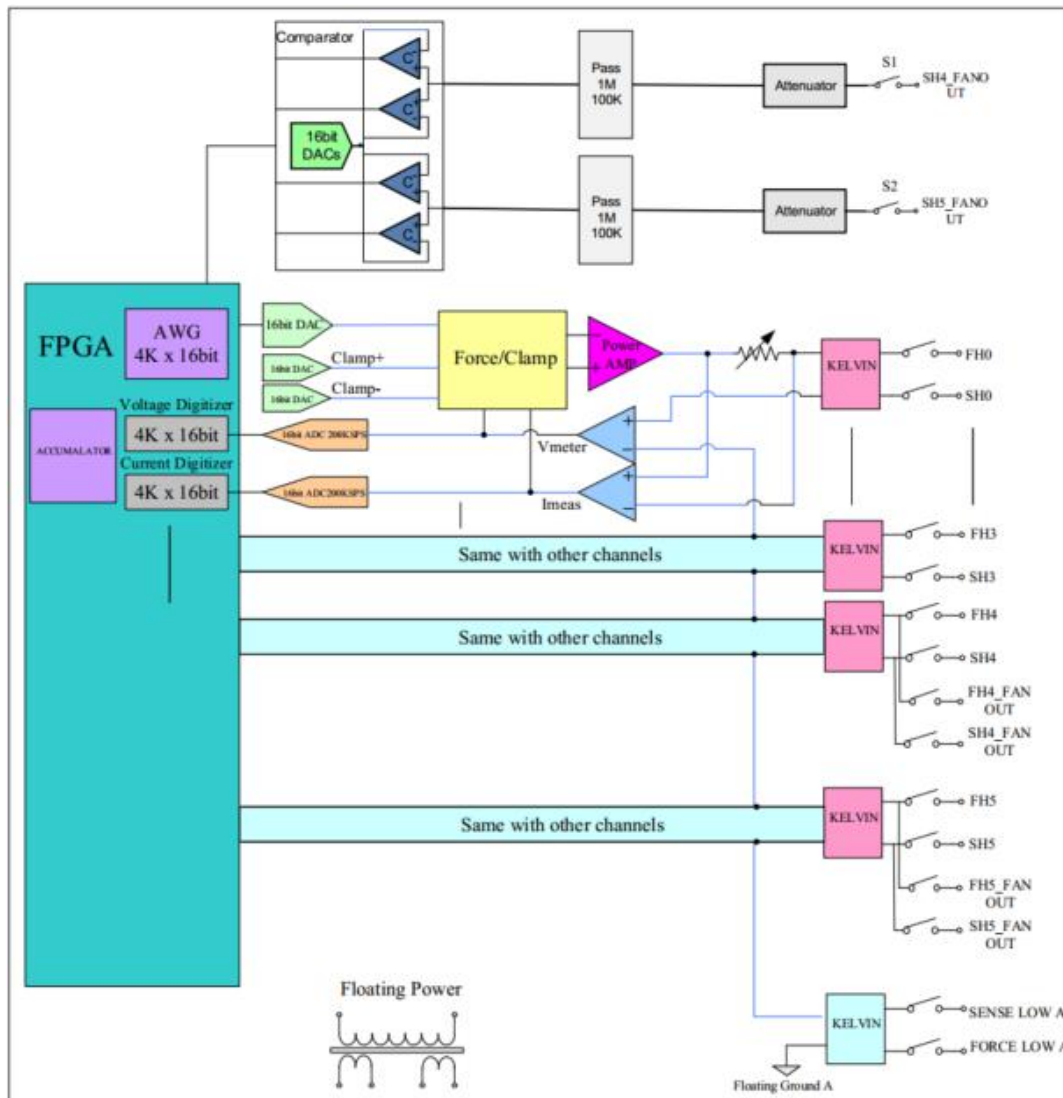
AWG	Resolution	Pattern Depth / Per Channel	DA Update Rate	AWG step
	16bit	4K	100KHz	20us
Digitizer	Resolution	Capture Memory / Per Channel	Maximum Sample Rate	
	16bit	4K	200kSPS	

• 指标

TMU	ch/board	2ch/bank, 4ch/board		
	Input voltage range	$\pm 30V/\pm 10V/\pm 3.6V$		
	Input impedance	1M		
	Trigger level accuracy	1%FSR		
	Input analog filter	100K/1M/PASS		
	Measuring Time Range	Manual Range		
		30ns~950ns	800ns~4ms	30ns~5.368s
	Time measuring accuracy	$\pm(30ns+0.1\%Rd)$		

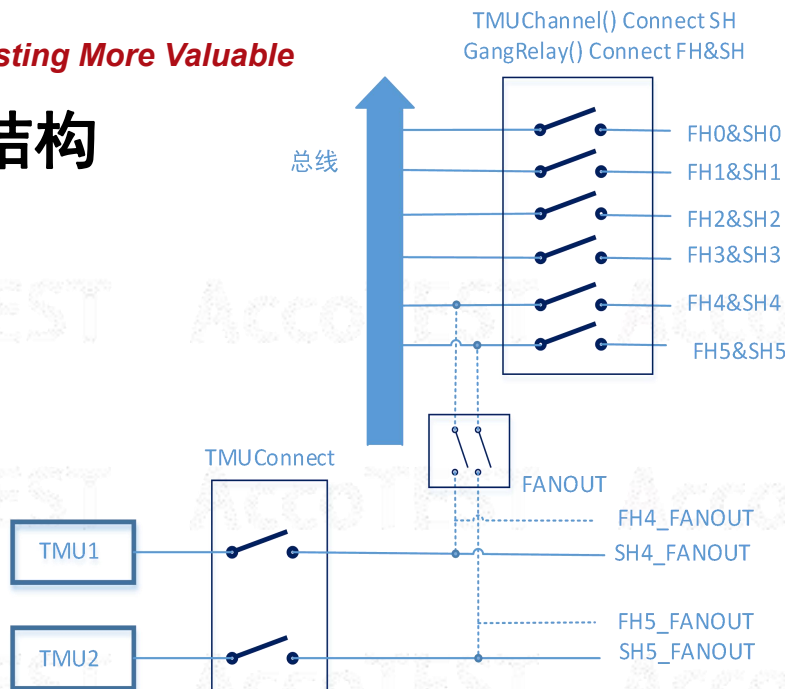
DVM	Range	Percent(%)	Offset(mV)
	3.6V	0.01% Rdg	0.3
	Digitizer resolution	Capture Memory	Maximum Sample Rate
	16bit	4K	200KSPS

• 原理框图



FXV1e Block Diagram-1 bank of 2

• 原理框图-FXVle总线结构



FANOUT:

目前仅支持CH4，CH5通道FANOUT至CH4_FANOUT, CH5_FANOUT的用户端；

GANG(exFANOUT):

通过总线方式，将任意两路或两路以上的FXVle通道（仅支持同一BANK内的通道）并联到主通道输出，用于拓展主通道的电流能力；

TMU:

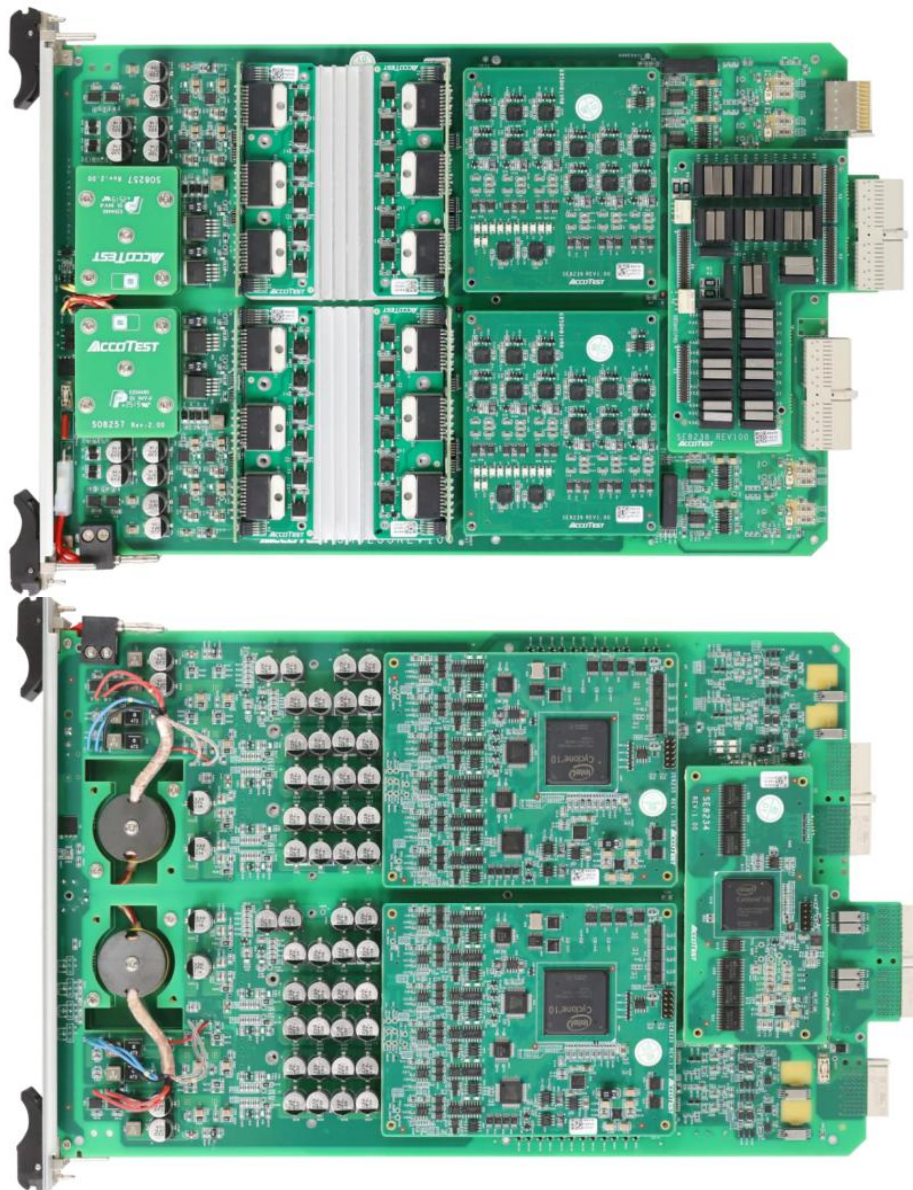
TMU1, TMU2为两个独立的时间测量单元，单独可用来测量Freq/Dutycycle/Tr/Tf/Tpluse/事件计数以及Tpd。

1. 常规用法：通过TMUConnect函数将TMU1，TMU2接通到SH4_FANOUT，SH5_FANOUT通道使用，使用时会占用CH4, CH5的SH_FANOUT通道，但不影响CH4，CH5通道的VI源功能；

2. 总线方式：将任意一路CH0~CH5通道，通过TMUchannel函数，通过总线指定给TMU1或者TMU2，可用来测量接入的单信号源的时间量（此时建议接入的通道设置为FI=0的状态，总线方式也会占用SH4/SH5_FANOUT通道），由于只有一条总线，不支持用总线模式测量CH0~CH3之间任意两通道的传输延时（用总线测量传输延时，需将其中一路接入SH4/SH5_FANOUT通道）；

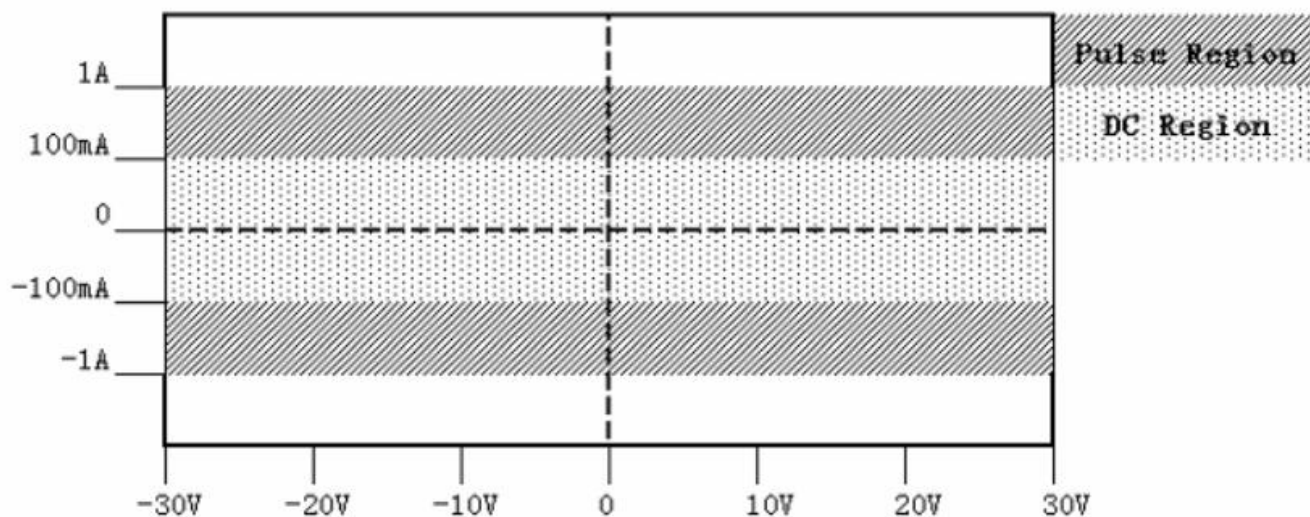
注意：GANG(exFANOUT)功能、TMU功能均需占用总线，不能同时使用，使用完之后建议通过GangReset/TMUChannelReset函数重置。

- 外观结构



• 四象限输出能力

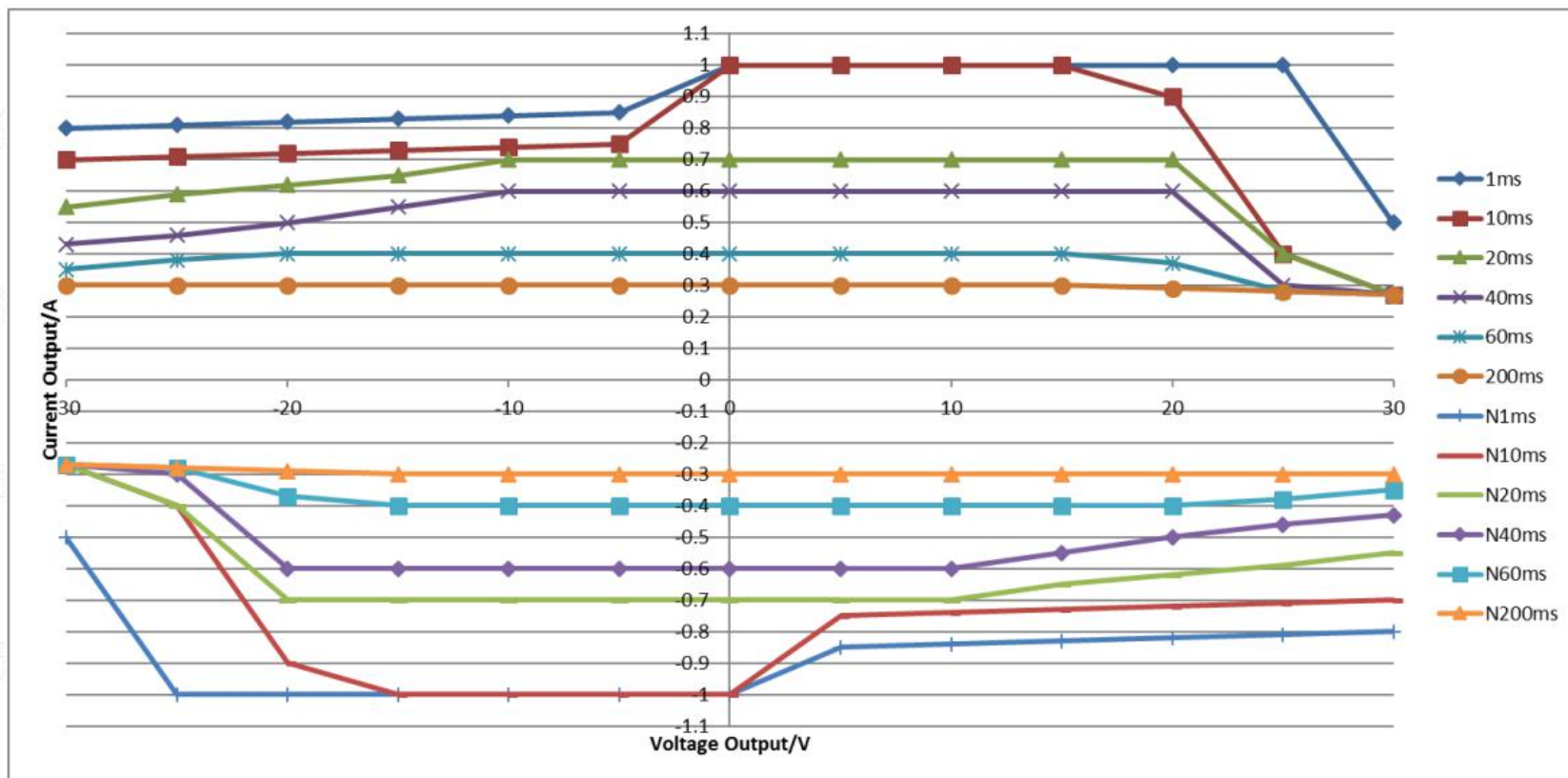
FXVIe 每通道在全电压范围内可输出 100mA 直流及1A 脉冲电流，四象限电压电流输出范围如图所示。



FXVIe 四象限电压电流输出范围

• 脉冲输出功率曲线

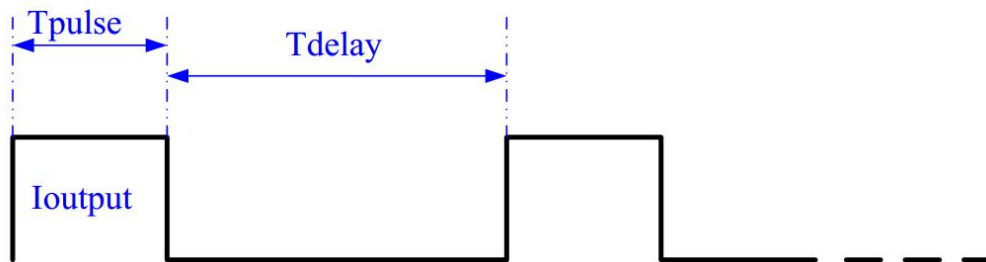
当输出电流大于 100mA 时，在编程时请参考输出功率曲线范围，并正确地调整输出占空比，以确保加在被测器件上的输出电压及输出电流正确。



FXVIe 1A 档脉冲输出功率曲线

• 脉冲输出功率曲线

大电流脉冲工作模式下依靠电容储能，在脉冲大电流后，会依靠 200mA 的恒流源充电，直至电容充满。因此连续大电流输出时，需要合理控制输出占空比以确保电容储能可以恢复。用户在脉冲工作模式下，可以按下式估算脉冲输出后的等待时间：



FXVIe 脉冲工作模式下时间设置

$$T_{delay} = T_{pulse} * I_{output} / 0.2$$

其中：

I_{output} 为脉冲电流值，单位为 A；

T_{pulse} 为电流脉冲持续时间，单位为 s；

T_{delay} 为脉冲输出后电容储能恢复所需的等待时间，单位为 s。

• 脉冲输出功率曲线

大电流脉冲工作模式下依靠电容储能，脉冲电流的持续时间与储能电容容值和输出电压值等因素有关，其近似计算公式如下所示。

$$t = \frac{[36 - (V_{out} + I_{out} * 3.5)] * 440 * 1e - 6}{I_{out} - 0.25}$$

其中：

I_{out} 为脉冲电流值，单位为 A；

t 为电流脉冲持续时间，单位为 s；

V_{out} 为输出电压值，单位为 V。

槽位及对应PogoPin定义

FXVIe对应的Analog槽位（TIB类型相关）

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	CBITE
	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	Cal
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A28D4 Config

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CBITE
	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CAL
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A24D8 Config

• 槽位及对应PogoPin定义

From PogoPin Tool

P101 slot6slot5

	AGND	G_11 FANOUT	SH_11 FANOUT	FH_11 FANOUT	SL_6-11	FL_6-11	FL_6-11	G_10 FANOUT	SH_10 FANOUT	FH_10 FANOUT	
	AGND	G_11	SH_11	FH_11	SL_6-11	FL_6-11	FL_6-11	G_10	SH_10	FH_10	
G_9	SH_9	FH_9	G_8	SH_8	FH_8	G_7	SH_7	FH_7	G_6	SH_6	FH_6
					CH_3-	G_5 FANOUT	SH_5 FANOUT	FH_5 FANOUT	G_3	SH_3	FH_3
					CH_3+	G_5	SH_5	FH_5	G_2	SH_2	FH_2
						SL_0-5	FL_0-5	FL_0-5	SL_0-5	FL_0-5	FL_0-5
						G_4 FANOUT	SH_4 FANOUT	FH_4 FANOUT	G_1	SH_1	FH_1
						G_4	SH_4	FH_4	G_0	SH_0	FH_0
		CH_2-			CH_2+			CH_1-			CH_1+
	AGND									CH_0-	
	AGND									CH_0+	
12	11	10	9	8	7	6	5	4	3	2	1

Open Config slot

Resource:

FXVIe

Slot:

slot6

Serial NO:

4_FAMOUT

☒ AutoSwitch

Config File Path:

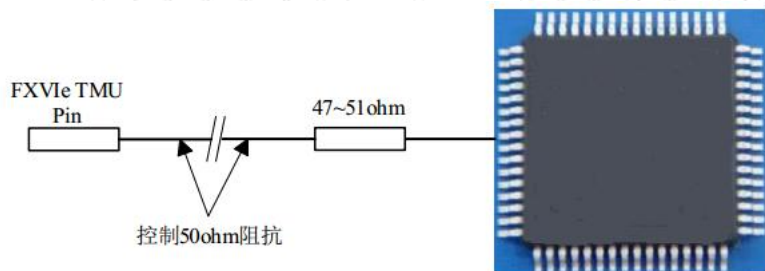
C:/AccoTentaa/AccoTEST System/ConfigEx.slot

Cell Channel And Slot:

P202	P201	P200	P109	P108	P107	P106	P105	P104	P103
P102									P101
P102									P101
P104									P103
P106									P105
P108	P103	P104	P105	P106	P107	P108	P109	P100	P101

• 原理图设计、PCB布局及注意事项

1. 同一BANK必须分配给同一工位；
2. FXVle TMU 通道的输入信号在 layout 时都需考虑 50ohm 阻抗匹配，另外，建议在器件输出管脚近端串接一个 47~51ohm 电阻，如图所示



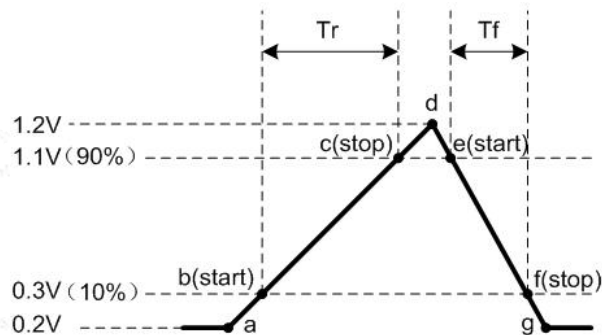
3. 需要测量传输延时的两个管脚所用的TMU通道至少有一个接SH4_FANOUT/SH5_FANOUT或者SH10_FANOUT/SH11_FANOUT接口；
4. 滞回宽度

FXVle TMU 30V 档的滞回宽度为 300mV;
 FXVle TMU 10V 档的滞回宽度为 100mV;
 FXVle TMU 3.6V 档的滞回宽度为 36mV;

For example, use 10V range to measure T_r/T_f :

Rise time(T_r) requirement: $V_b - V_a > 100\text{mV}$, $V_b < V_c < V_d$.

Rise time(T_f) requirement: $V_d - V_e > 100\text{mV}$, $V_g < V_f < V_e$.



5. 使用Gang, TMU, FANOUT功能时, 参考<总线框图>的说明。

FXVle 单板包含四路时间测量单元，每组两路，分别对应到 FANOUT 通道的信号接口。
 即 SH4_FANOUT、SH5_FANOUT、SH10_FANOUT、SH11_FANOUT

• VI源功能（Turbo, FANOUT, exFANOUT）

ContackCheck()

GetContackCheck()

Set()

Tset()

SetSyn()

SetClamp()

MeasureVI()

GetMeasResult()

BlockRead()

Pulse()

SetFLOff()

GangRelay()

GangSet()

GangSetSyn()

GangMeasure()

GangGetMeasResult()

GangBlockRead()

GangReset()

• FXVle - 接触电阻检测

ContactCheck (FXVle_CONTACTMODE *checkMode*) ;

Remarks

设置高低端接触检测的模式。

注意:

- 1) 通过该函数对高低端电路进行接触检测，检测结束后， VI 源将恢复至检测之前的状态；
- 2) 该函数支持 Group 操作。

Parameters

checkMode

接触测试的模式，可设置为：

FXVle_HIGH_SIDE 检测高端电路的连接

FXVle_LOW_SIDE 检测低端电路的连接

```
fxvie0.ContactCheck(FXVle_HIGH_SIDE); // set the contact check mode to high side
```

• FXVle - 接触电阻检测

```
GetContactCheckResult (UINT siteCount, double&highRes,  
double&lowRes);
```

Remarks

读取该工位接触检测的结果。

Parameters

siteCount UINT 型变量，选择读取接触检测结果的工位。 0 代表 Site1， 1 代表 Site2.....

highRes double 型变量，输出高端检测的电阻值。

lowRes double 型变量，输出低端检测的电阻值。

Return Values

返回-1: 该工位无效

返回 0: 该工位接触检测 Pass

返回 1: 该工位接触检测高端 Fail

返回 2: 该工位接触检测低端 Fail

```
double highRes = 0;  
double lowRes = 0;  
int result = fxvie0.GetContactCheckResult(0, highRes, lowRes); // Get contact check result ofSITE_1
```

• FXVle –VI源/FANOUT

```
Set (VIMode viMode, double setValue, FXVle_VRNG vRange,
FXVle_IRNG iRange, FXVle_OUT_RELAY relayStatus =
FXVle_RELAY_HOLD, double risingTime = 0.02);
```

Remarks

设置 FXVle 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Parameters

viMode FXVle 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-30~30。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

vRange FXVle 电压量程。

FXVle_30V

FXVle_10V

FXVle_3p6V

iRange FXVle 电流量程。

FXVle_1A

FXVle_100MA

FXVle_10MA

FXVle_1mA

FXVle_100UA

FXVle_10UA

• FXVle –VI源/FANOUT

Parameters

relayStatus 设置输出继电器如何动作，分为七种情况：

FXVle_RELAY_HOLD：输出继电器保持上一个状态；

FXVle_RELAY_ON：打开输出继电器；

FXVle_RELAY_FANOUT_ON：打开扇出继电器；

FXVle_RELAY_ALL_ON：打开输出和扇出继电器；

FXVle_RELAY_OFF：关闭输出和扇出继电器；

FXVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FXVle 处于独立电压表模式。

FXVle_RELAY_SENSE_FANOUT_ON：启动独立测量模式。只打开 SENSE 线扇出继电器，关闭 FORCE 线扇出继电器。此时 FXVle 处于独立电压表模式。

risingTime

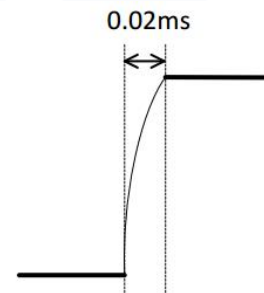
double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。该参数不写，则默认为 0.02mS；

1) 该参数设置小于等于 0.02mS 时， *risingTime*=0.02mS，为 Normal 模式；

2) 该参数设置小于等于 0.04mS 时， *risingTime*=0.04mS，为 Slow 模式；

3) 该参数设置大于 0.04mS 时，自动启动 Capload 功能， *risingTime* 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

```
fxvie0.Set(FV, 5, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
```



• FXVle –Turbo

```
TSet(VIMode viMode, double setValue, FXVle_VRNG vRange,  
FXVle_IRNG iRange, FXVle_OUT_RELAY relayStatus=FXVle_RELAY_HOLD,  
double risingTime = 0.02);
```

Remarks

以 Turbo 模式设置 FXVle 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间，配合 STSTArming 函数保证状态设置完成。

注意:

1)该函数与 Set 函数的区别是:

a. Set 函数执行结束时，对应通道已达到目标状态，而 TSet 函数只是设置了对应通道的目标状态，需要配合 STSTArming 函数等待状态设置完成;

b. Set 函数在进行 Group 操作时，要求所有通道的 VI 源模式，电压/电流量程，继电器状态保持一致，而 TSet 函数进行 Group 操作时没有这样的要求;

2)不同通道间设置状态时可依次使用 TSet 函数，增加一条 STSTArming 函数等待所有状态设置完成，可达到节省测试时间的目的;

3)使用该函数要求软件为 AccoTestSystemVersion2100build200814_r p2.0 及以上版本，要求 FXVle 板卡 B 模块的逻辑为 0x12 及以上版本。

Parameters

viMode FXVle 工作模式，有两种模式:

FV: 选择恒压模式

FI: 选择恒流模式

setValue double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-30~30。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

• FXVle –Turbo

vRange FXVle 电压量程。

FXVle_30V FXVle_10V FXVle_3p6V

iRange FXVle 电流量程。

FXVle_1A FXVle_100MA FXVle_10MA

FXVle_1mA FXVle_100UA FXVle_10UA

relayStatus 设置输出继电器如何动作，分为七种情况：

FXVle_RELAY_HOLD：输出继电器保持上一个状态；

FXVle_RELAY_ON：打开输出继电器；

FXVle_RELAY_FANOUT_ON：打开扇出继电器；

FXVle_RELAY_ALL_ON：打开输出和扇出继电器；

FXVle_RELAY_OFF：关闭输出和扇出继电器；

FXVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FXVle 处于独立电压表模式。

FXVle_RELAY_SENSE_FANOUT_ON：启动独立测量模式。只打开 SENSE 线扇出继电器，关闭 FORCE 线扇出继电器。此时 FXVle 处于独立电压表模式。

risingTime double 型变量，设置变化时间。单位为mS，分辨率为0.01mS。该参数不写，则默认为0.02mS；

1) 该参数设置小于等于 0.02mS 时， risingTime=0.02mS，为 Normal 模式；

2) 该参数设置小于等于 0.04mS 时， risingTime=0.04mS，为 Slow 模式；

3) 该参数设置大于 0.04mS 时，自动启动 Capload 功能， risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

• FXVle –Turbo

例1: Turbo 模式下, 设置 fxvie0 在恒压模式下, 使用5V/100MA 档位, 输出4V电压, risingTime 使用默认值; 设置fxvie1 在恒压模式下, 使用2V/10MA 档位, 输出 1.5V 电压, risingTime 使用默认值。

```
fxvie0.TSet(FV, 4, FXVle_5V, FXVle_100MA, FXVle_RELAY_ON);  
fxvie1.TSet(FV, 1.5, FXVle_2V, FXVle_10MA, FXVle_RELAY_ON);  
STSTArming();
```

例2: fxvie0 在恒压模式下, 使用5V/100MA 档位, 输出4V 电压, risingTime 使用默认值; fxvie1 在恒压模式下, 使用2V/10MA 档位, 输出1.5V 电压, risingTime使用默认值; 对这两个通道进行Group 操作 (gpfxvie), 目标状态设置为恒压为 0, 10V/1MA 档位, RelayOFF。

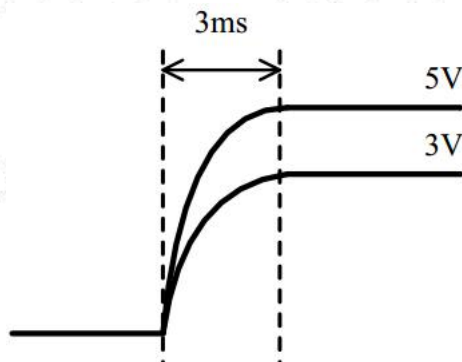
```
fxvie0.Set(FV, 4, FXVle_5V, FXVle_100MA, FXVle_RELAY_ON);  
fxvie1.Set(FV, 1.5, FXVle_2V, FXVle_10MA, FXVle_RELAY_ON);  
// Turbomodel  
gpfxvie.TSet(FV, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_OFF);  
STSTArming();
```

• FXVle

```
SetSyn(VIMode viMode, const double* setValue, UINT siteSize,
FXVle_VRNG vRange, FXVle_IRNGiRange,
FXVle_OUT_RELAY relayStatus = FXVle_RELAY_HOLD,
double risingTime = 0.02);
```

例2: fxvie0 通道的 SITE1 设置其输出 3V, SITE2 设置其输出 5V, 且要求 SITE1和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};
int siteSize = 2;
fxvie0.Set(FV, 0, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
delay_ms(1);
fxvie0.SetSyn(FV, setValue, siteSize, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON, 3);
```



- FXVle

SetClamp(double percent_PFS, double percent_NFS);

Remarks

设置 FXVle 的正负箝位值。

注意:

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 102%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围： 10~102

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围： 10~102

例 1：设置 fxvie1 为 FI=0 模式，钳位到 (50, 20)，然后切换 fxvie1 为 FV=1V 模式，钳位到(25, 25)。

```
fxvie1.Set(FI, 0, FXVle_10V, FXVle_10MA, FXVle_RELAY_ON);  
// positive clamp to 50% of full range(FXVle_10V), negative clamp to 20% of full range  
fxvie1.SetClamp(50, 20);  
// switch from FI to FV mode, then positive/negative clamps recover back to 110% of full range  
fxvie1.Set(FV, 0, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);  
// reset positive clamp to 25% of full range(FXVle_100MA), negative clamp to 25% of full range  
fxvie1.SetClamp(25, 25);  
// Set FV=1V  
fxvie1.Set(FV, 1, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
```

• FXVle

```
MeasureVI(UINT sampleTimes, double samplePeriod,  
FXVle_MI_GAIN miGain = FXVle_MI_X1,  
FLOATMEASMODE measMode = MEAS_NORMAL,  
UINT T1 = 0, UINT T2 = 0, UINT T3 = 0, UINT T4 = 0);
```

Remarks

设置 FXVle 的测量次数、测量间隔、测流增益、测量模式以及 AWG 测量模式下的四次测量启动时刻

Parameters

sampleTimes 设置 AD 采样次数。范围 1~4096

注意:

当 $sampleTimes < 1$ 时, $sampleTimes = 1$;

当 $sampleTimes > 4096$ 时, $sampleTimes = 4096$ 。

samplePeriod 设置每次采样之间的时间间隔, 单位uS。最小采样间隔为5uS, 分辨率1uS。范围:5~30000。

注意:

当 $samplePeriod < 5$ 时, $samplePeriod = 5$;

当 $samplePeriod > 30000$ 时, $samplePeriod = 30000$ 。

miGain FXVle 测流增益。

FXVle_MI_X1 **FXVle_MI_X10**

该参数可以不写, 默认值为 FXVle_MI_X1。

measMode 测量模式选择, 可设置为:

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG 测量模式。只做设置, 运行 AWG 时启动。

该参数可以不写, 默认值为 MEAS_NORMAL 模式

- FXVIe

Parameters

T1、T2、T3、T4

单位 uS。分辨率 10uS，范围：0 ~ (T1+T2+T3+T4) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、T2、T3、T4 为相对时间。即 T2 的启动时间为相对 T1 的启动时间，T3 的启动时间为相对 T2 的启动时间，T4 的启动时间为相对 T3 的启动时间。

例 1：设置 fxvie0 为普通模式测量，采样 20 次，间隔 10uS。

```
fxvie0.MeasureVI(20,10);
```

• FXVIe

```
GetMeasResult(UINT siteCount, MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT, BYTE triggerNum = 0);
```

Remarks

获取 MeasureVI()函数的测量结果。

1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；

2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

1) FXVIe 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET或者 MIRET 即可。

2) 当参数 sampleNumber 选择 TRIG_RESULT 时，如果没有发生触发，则函数返回值为-3，如果测量模式与回读模式不符，则函数返回值为-5。

• FXVIe

Parameters

siteCount UINT 型变量，选择测量数据工位。 0 代表 Site1， 1 代表 Site2.....

retype 选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam} - 1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum 触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读第一次发生触发的位置。该参数设置为 1，表示回读第二次发生触发的位置

- FXVle

例 1：返回 fxvie0 电流测量结果平均值。

```
// fxvie0 sampleTimes=20, samplePeriod=10uS
fxvie0.MeasureVI(20, 10);
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value of current
    adresult[i] = fxvie0.GetMeasResult(i, MIRET);
}
```

例 2：返回 fxvie0 电压测量结果第 15 个点的值。

```
//fxvie0 sampleTimes=20, samplePeriod=10uS
fxvie0.MeasureVI(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=fxvie0.GetMeasResult(i, MVRET, 14);
}
```

• FXVle

```
BlockRead(UINT siteCount,UINT start,UINT size, double* buffer,  
MeasRet retType = MVRET);
```

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Parameters

siteCount UINT 型变量，选择测量数据工位。 0 代表 Site1， 1 代表 Site2.....

start UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-start）

buffer 存放数据结果的数组，由用户定义，如： `double result[1000]={0.0};`

注意：

- 1) *buffer* 数组长度需不小于 *size* 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retype 选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

```
double result[SITENUM][200] = { 0.0 };  
fxvie0.MeasureVI(200, 10);  
for (i = 0; i < SITENUM; i++)  
{  
    // start=0; size=200; Get MVRET, saved to result.  
    fxvie0.BlockRead(i, 0, 200, result[i], MVRET);  
}
```

• FXVle

```
Pulse(double pulseValue, UINTpulseTime = 300, int measStartTime  
= 0, UINTsampleTimes= 10, double sampleInterval= 30);
```

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意:

Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Parameters

pulseValue double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-30~30；

选择 FI 模式时，单位 A。最大范围为-1~1。

pulseTime UINT 型变量，设置脉冲时间。单位 uS。分辨率： 10uS。范围： 300~40000。

可以不写，默认为 300uS。

measStartTime int 型变量，设置启动测量时间，单位 uS。范围： -60000~60000。可以不写，默认为 0。

sampleTimes UINT 型变量，设置采样次数。范围 1~4096。可以不写，默认为 10。

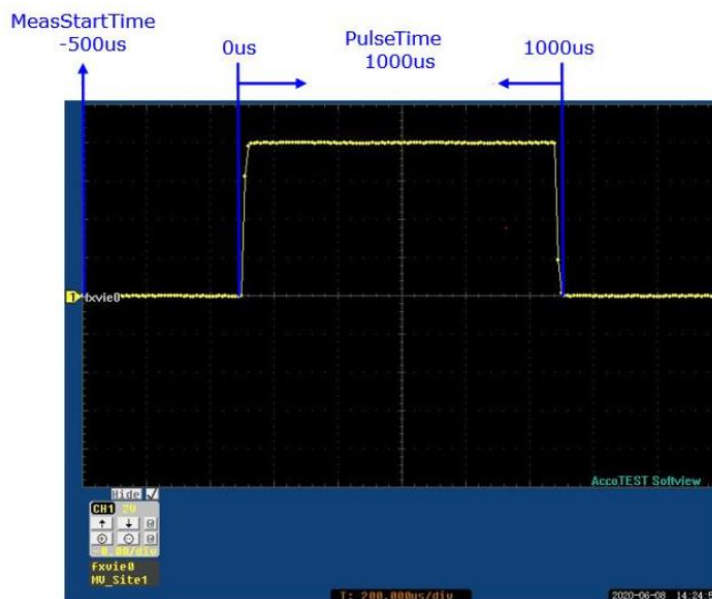
sampleInterval UINT 型变量，设置采样间隔时间，单位 uS。范围： 5~30000。最小采样间隔为 5uS，
分辨率 1uS。可以不写，默认为 30uS。

• FXVle

例 1：设置 fxvie0 起始状态 FV 为 0。设置 fxvie0 输出 1mS 的 8V 脉冲；采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
fxvie0.Set(FV, 0, FXVle_10V, FXVle_10MA, FXVle_RELAY_ON);
// first set fxvie0 in FV=0 mode,
fxvie0.Pulse (8,1000, -500, 200, 10);
//pulse output value=8v, pulse duration=1000uS, start time for measuring is -500uS
//sample=200,interval time=10uS
```

使用 SoftView 软件抓取的波形右图：



- FXVle

SetFLOff ();

Remarks

断开当前通道所在 bank 的 ForceLow 继电器。

fxvie0.SetFLOff ()

• FXVle –exFANOUT 通道并联

```
GangRelay(FXVle Parallel_CH0, FXVle Parallel_CH1,  
FXVle Parallel_CH2, FXVle Parallel_CH3, FXVle Parallel_CH4);
```

Remarks

设置通道并联，将选定的通道并联到一起。

注意:

- 1) 该函数不支持 group;
- 2) 并联通道属性必须都是多工位或都是 NOSITE;
- 3) 相同工位下，所有并联通道必须在同一 bank;
- 4) 不同工位下，所有通道必须在不同 bank;
- 5) 对于相同 bank 的任一通道，使用该函数后，若未进行复位操作，则该 bank 所有通道的 TMUChannel 函数设置无效;
- 6) 对于并联通道，使用该函数后，若未进行复位操作，则 Set 函数设置无效;
- 7) 该函数设置后，所有并联通道状态为 FI 为 0，电压档位 30V，电流档位 100UA。

Parameters

Parallel_CHx FXVle 类型变量，并联通道 x，将该通道并联到主通道上

• FXVle –exFANOUT 通道并联（正确示例）

例 1：设置 fxvie1 和 fxvie2 通道并联到 fxvie0 通道上。复位后，方可进行 TMUChannel操作或 Set 操作。

```
FXVle fxvie0("S18_0,S18_6", "fxvie0");  
FXVle fxvie1("S18_1,S18_7", "fxvie1");  
FXVle fxvie2("S18_2,S18_8", "fxvie2");  
FXVle fxvie3("S18_3,S18_9", "fxvie3");  
FXVle fxvie4("S18_4,S18_10", "fxvie4");  
FXVle fxvie5("S18_5,S18_11", "fxvie5");
```

```
STSSetMultiSiteBind(MD_FXVle,SITE_1,"S18_0,S18_1,S18_2,S18_3,S18_4,S18_5");  
STSSetMultiSiteBind(MD_FXVle,SITE_2,"S18_6,S18_7,S18_8,S18_9,S18_10,S18_11");
```

```
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangReset();
```

```
fxvie4.TMUChannel(fxvie0);  
fxvie4.TMUChannelReset();
```

```
fxvie0.Set(FV, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_OFF);
```

• FXVle -exFANOUT 通道并联（错误示例）

对应注意事项 2), fxvie0 和 fxvie1 的通道属性为多工位, fxvie2 属性为 NOSITE。

```
FXVle fxvie0("S18_0", "fxvie0");  
FXVle fxvie1("S18_1", "fxvie1");  
FXVle fxvie2("S18_5", "fxvie2");  
STSSetMultiSiteBind(MD_FXVle, SITE_1, "S18_0, S18_1");  
STSSetMultiSiteBind(MD_FXVle, NO_SITE, "S18_5");  
fxvie0.GangRelay(fxvie1, fxvie2);
```

错误示例1

对应注意事项 3), S18_8 和 S18_1 不属于相同的 bank。

```
FXVle fxvie0("S18_2, S18_8", "fxvie0");  
FXVle fxvie1("S18_0, S18_1", "fxvie1");  
FXVle fxvie2("S18_3, S18_9", "fxvie2");  
STSSetMultiSiteBind(MD_FXVle, SITE_1, "S18_0, S18_2, S18_3");  
STSSetMultiSiteBind(MD_FXVle, SITE_2, "S18_1, S18_8, S18_9");  
fxvie0.GangRelay(fxvie1, fxvie2);
```

错误示例2

对应注意事项 4), S18_0 和 S18_3、S18_1 和 S18_4、S18_2 和 S18_5 属于相同的 bank。

```
FXVle fxvie0("S18_0, S18_3", "fxvie0");  
FXVle fxvie1("S18_1, S18_4", "fxvie1");  
FXVle fxvie2("S18_2, S18_5", "fxvie2");  
STSSetMultiSiteBind(MD_FXVle, SITE_1, "S18_0, S18_1, S18_2");  
STSSetMultiSiteBind(MD_FXVle, SITE_2, "S18_3, S18_4, S18_5");  
fxvie0.GangRelay(fxvie1, fxvie2);
```

错误示例3

• FXVle –exFANOUT 通道并联

```
GangSet(double setValue, double risingTime = 0.02);
```

Remarks

设置通道电流并联功能，包括电流值及 Capload 时间。

注意:

- 1) 该函数不支持 group;
- 2) 该函数必须在 GangRelay 和 GangReset 函数之间使用。
- 3) 该函数设置后，所有并联通道电压档位 30V，电流档位 1A。

Parameters

setValue double 型变量，设置恒流值。单位为 A。最大范围为-1.2~1.2。

risingTime double 型变量，设置变化时间。单位为mS，分辨率为0.01mS。

该参数不写，则默认为0.02mS；最大范围为 0.02~65。

- 1) “该参数/并联通道个数” 后，其值小于等于 0.02mS 时， *risingTime*=0.02mS，为 Normal 模式；
- 2) “该参数/并联通道个数” 后，其值小于等于 0.04mS 时， *risingTime*=0.04mS，为 Slow 模式；
- 3) “该参数/并联通道个数” 后，其值大于 0.04mS 时，自动启动 Capload 功能，*risingTime* 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定， Capload 的最大值为 65mS。

例 1：设置 fxvie1 和 fxvie2 通道并联到 fxvie0 通道上，输出电流为 1A，*risingTime* 默认为 0.02mS。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangSet(1);  
fxvie0.GangReset()
```

• FXVle -exFANOUT 通道并联

```
GangSetSyn(const double* setValue, UINT siteSize,  
double risingTime = 0.02);
```

Remarks

设置通道电流并联功能， 对不同的工位设定不同的电流值， 且该操作对不同工位完全同步。

注意：

- 1) 该函数不支持 group;
- 2) 该函数必须在 GangRelay 和 GangReset 函数之间使用。
- 3) 该函数设置后，所有并联通道电压档位 30V，电流档位1A。

Parameters

setValue double 型变量，设置恒流值。 单位为 A。最大范围为-1.2~1.2。

siteSize UINT 型变量，设置执行同步操作的工位数。若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、 SITE3、 SITE4。

注意：

- 1) *setValue* 数组长度必须大于 *siteSize* 的值，否则可能会发生不可预期的错误；
- 2) 若 *siteSize* 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
- 3) 若 *siteSize* 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

risingTime double 型变量，设置变化时间。单位为 mS，分辨率为 0.01mS。

该参数不写，则默认为 0.02mS；最大范围为 0.02~65。

- 1) “该参数/并联通道个数” 后，其值小于等于 0.02mS 时， *risingTime*=0.02mS，为 Normal 模式；
- 2) “该参数/并联通道个数” 后，其值小于等于 0.04mS 时， *risingTime*=0.04mS，为Slow 模式；
- 3) “该参数/并联通道个数” 后，其值大于 0.04mS 时，自动启动 Capload 功能，*risingTime* 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定， Capload 的最大值为65mS。

- FXVle –exFANOUT 通道并联

例1：设置 fxvie1 和 fxvie2 通道并联到 fxvie0 通道上，SITE1 设置其输出电流为 1A，SITE2 设置其输出 1.2A，risingTime 默认为 0.02mS。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
double setValue[2] = { 1, 1.2 };  
int siteSize = 2;  
fxvie0.GangSetSyn(setValue, siteSize)  
fxvie0.GangReset();
```

• FXVle –exFANOUT 通道并联

GangMeasure (UINT *sampleTimes*, double *samplePeriod*) ;

Remarks

设置 FXVle 并联模式下的测量次数、测量间隔。

注意:

- 1) 该函数不支持 group;
- 2) 该函数必须在 GangRelay 和 GangReset 函数之间使用。

Parameters

sampleTimes 设置 AD 采样次数。范围 1~4096

注意:

当 $sampleTimes < 1$ 时, $sampleTimes = 1$;

当 $sampleTimes > 4096$ 时, $sampleTimes = 4096$ 。

samplePeriod 设置每次采样之间的时间间隔, 单位 uS。

最小采样间隔为 5uS, 分辨率 1uS。范围: 5~30000。

注意:

当 $samplePeriod < 5$ 时, $samplePeriod = 5$;

当 $samplePeriod > 30000$ 时, $samplePeriod = 30000$ 。

例 1: 设置 fxvie0 采样 20 次, 间隔 10uS。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangSet(1);  
fxvie0.GangMeasure(20, 10);
```

• FXVle –exFANOUT 通道并联

```
GangGetMeasResult(UINT siteCount, MeasRetretType = MVRET,  
int sampleNumber = AVERAGE_RESULT);
```

Remarks

获取 GangMeasure()函数的测量结果。

1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；

2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意:

1) FXVle 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET或者 MIRET 即可。

2) 该函数必须在 GangRelay 和 GangReset 函数之间使用。

Parameters

siteCount UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

retype 选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 GangMeasure 的采样次数为 sam，则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

• FXVle –exFANOUT 通道并联

例 1：返回 fxvie0 电流测量结果平均值。

```
fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangSet(1);
delay_ms(1);
fxvie0.GangMeasure(20, 10); // fxvie0 sampleTimes=20, samplePeriod=10uS
for (i = 0; i < SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value of current
    adresult[i] = fxvie0.GangGetMeasResult(i, MIRET);
}
fxvie0.GangReset();
```

例 2：返回 fxvie0 电压测量结果第 15 个点的值

```
fxvie0.GangRelay(fxvie1, fxvie2);
fxvie0.GangSet(1);
delay_ms(1);
fxvie0.GangMeasure(20, 10); //fxvie0sampleTimes=20, samplePeriod=10uS
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=fxvie0.GangGetMeasResult (i, MVRET, 14);
}
fxvie0.GangReset();
```

• FXVle –exFANOUT 通道并联

```
GangBlockRead(UINT siteCount,UINT start,UINT size,  
double* buffer,MeasRetretType = MVRET);
```

Remarks

此函数读取并联测量时一个数据块的测量结果，数据块的起始地址和大小由用户定义。

注意：

该函数必须在 GangRelay 和 GangReset 函数之间使用。

Parameters

siteCount UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start UINT 型变量，数据块的起始地址，非负整数。

size UINT 型变量，数据块的大小，正整数。

buffer 存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

1) *buffer* 数组长度需不小于 *size* 的值，否则可能会发生不可预期的错误；

2) 建议把该数组定义成全局变量。

retype 选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

- FXVle –exFANOUT 通道并联

```
double result[SITENUM][200] = { 0.0 };  
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangSet(1);  
delay_ms(1);  
fxvie0.GangMeasure(200, 10);  
for (i = 0; i< SITENUM; i++)  
{  
    // start=0; size=200; Get MVRET, saved to result.  
    fxvie0.GangBlockRead(i, 0, 200, result[i], MVRET);  
}  
fxvie0.GangReset();
```

• FXVle –exFANOUT 通道并联

GangReset ();

Remarks

并联通道复位功能，将并联通道复位，并联模式解除。

恢复至恒压为 0，电压档位为 30V，电流档位为 1mA，继电器 RelayOff。

注意：

在进行其它非并联模式操作之前，必须先使用该函数解除并联模式。

例 1：将 fxvie0、 fxvie1 和 fxvie2 的并联模式解除。

```
fxvie0.GangRelay(fxvie1, fxvie2);  
fxvie0.GangSet(1);  
fxvie0.GangMeasure(20, 10);  
fxvie0.GangReset();
```

- FXVIe –AWG

AwgLoader()

AwgSelect()

AwgRun()

AwgStop()

AwgClear()

SetMeasVTrig()

SetMeasITrig()

• FXVle –AWG

```
AwgLoader(char* awgName, VIMode viMode, FXVle_VRNG vRange,  
FXVle_IRNG iRange, double* awgData, UINT awgSize);
```

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意:

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 FXVle 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 FXVle 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 FXVle 通道导入的多个 AWG 波形总长度小于4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 FXVle 通道导入的多个 AWG 波形总长度大于4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

• FXVle –AWG

Parameters

awgName char 型变量, AWG 波形名称。命名不区分大小写。

viMode FXVle 工作模式, 有两种模式:

FV 选择恒压模式

FI 选择恒流模式

vRange FXVle 电压量程。

FXVle_30V **FXVle_10V** **FXVle_3p6V**

iRange FXVle 电流量程。

FXVle_1A **FXVle_100MA** **FXVle_10MA**

FXVle_1mA **FXVle_100UA** **FXVle_10UA**

awgData double 型指针, 存储 AWG 波形数据。

awgSize UINT 型变量, 设置 AWG 波形数据的大小。该参数范围为 1~4096。

例1: 建立一个波形数据组, fxvie0 从16.5V 上升到21.5V, 步距0.1V。

```
int sam = 50; //AWG waveform data length
int interval = 200; //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};
STSAWGCreateRampData(&awg_pattern[0],sam,1,16.5,21.5);
//pattern array saved in 'awg_pattern', start from 16.5 to 21.5
fxvie0.AwgLoader("Vst ", FV, FXVle_30V, FXVle_100MA, awg_pattern, sam);
```

- FXVle –AWG

例2：建立一个波形数据组，fxvie0 输出频率为100Hz、峰峰值为4V、直流偏置为10V 的正弦波。

```
int sam = 1000; //data size for AWG pattern
int interval = 10; //interval time for AWG pattern
double awg_pattern[2000]={0.0};
STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0);
//set a sinewave data array of 100Hz, Vpp=4, DC Offset=10
fxvie0.AwgLoader("Sine", FV, FXVle_30V, FXVle_100MA, awg_pattern, sam);
// pattern named as 'SINE', FV mode
```

• FXVle –AWG

```
AwgSelect(char* awgName, UINT startAddr, UINT stopAddr,  
int loopBackAddr, double awgInterval= 10.0);
```

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 FXVle 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRun 才会有 AWG 波形输出。

Parameters

awgName char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr UINT 型变量，设置 AWG 波形起始点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr UINT 型变量，设置 AWG 波形结束点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；

当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。

数据范围为 10~60000。默认值为 10uS。

• FXVle –AWG

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200; // interval time for AWG pattern, unit is uS
int sam1 = 1000; //data size for AWG pattern
int sam2 = 500; // data size for AWG pattern
double awg_pattern1[1000]={0.0}; //define AWG_Vst data array
double awg_pattern2[500]={0.0}; //define AWG_Vuvlo data array

//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0], sam1, 1, 16.5, 21.5);
// set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0], sam2, 1, 16.5, 10.5);

// pattern named as 'AWG_Vst', FV mode
fxvie0.AwgLoader("AWG_Vst", FV, FXVle_30V, FXVle_100MA, awg_pattern1, sam1);
// pattern named as 'AWG_Vuvlo', FV mode
fxvie0.AwgLoader("AWG_Vuvlo", FV, FXVle_10V, FXVle_100MA, awg_pattern2, sam2);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of its 'AwgLoader()'
fxvie0.Set(FV, 16.5, FXVle_30V, FXVle_100MA, FXVle_RELAY_ON);

// Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
fxvie0.AwgSelect("AWG_Vst", 0, sam1 - 1, 0, interval);
STSAWGRun(); //AWG synchronous running started for pattern of 'AWG_Vst'
```

• FXVle –AWG

```
AwgRun(char* awgName, UINT startAddr, UINT stopAddr,  
int loopBackAddr, double awgInterval= 10.0,  
BOOL runMode = AWG_SINGLE, UINT delayTime_ms = 0);
```

Remarks

某一路 FXVle 通道 AWG 单独启动函数。

注意:

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRun。

• FXVle –AWG

Parameters

awgName char 型变量， AWG 波形名称。应与之前通过 *AwgLoader* 函数导入的 AWG 波形名称保持一致。

startAddr UINT 型变量，设置 AWG 波形起始点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr UINT 型变量，设置 AWG 波形结束点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时， AWG 波形保持在运行结束的位置；

当该值在 0~sam-1 时， AWG 波形运行结束后，会保持在设定点的位置。

awgInterval double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为10~60000。

该参数可以不写，默认 10uS。

runMode 运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 *AwgStop()*函数停止。

默认值为 **AWG_SINGLE**。

delayTime_ms UINT 型变量，单位 mS。范围： 0~60000。设置延时时间。

AWG 输出*delayTime_ms* 之后，系统允许*AwgRun* 之后的操作开始运行。

delayTime_ms 不填的时候，*delayTime_ms* 默认为AWG 扫描时间，只有在AWG 扫描完毕之后，系统才允许*AwgRun* 之后的操作开始运行。

注意：

建议不填该参数！

- FXVle –AWG

例1: fxvie0 连续输出 120Hz 正弦波, 幅值 0.33V, 直流偏置为 3.3V。

```
int sam = 830; //size of AWG pattern
double interval = 10; //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0], sam, 1, 0.66, 3.3, 0);
// pattern named as 'Sin_AWG', FV mode
fxvie0.AwgLoader("Sin_AWG", FV, FXVle_10V, FXVle_1A, awg_pattern, sam);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its 'AwgLoader()'
fxvie0.Set(FV, 0, FXVle_10V, FXVle_1A, FXVle_RELAY_ON);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
fxvie0.AwgRun("Sin_AWG ", 0, (sam - 1), 0, interval, AWG_LOOP);
```

• FXVle –AWG

AwgStop(void);

Remarks

停止某一路 FXVle 通道的 AWG 循环运行。

例1: fxvie0 生成一个周期为1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150; //data size of AWG pattern
double interval = 10; //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};

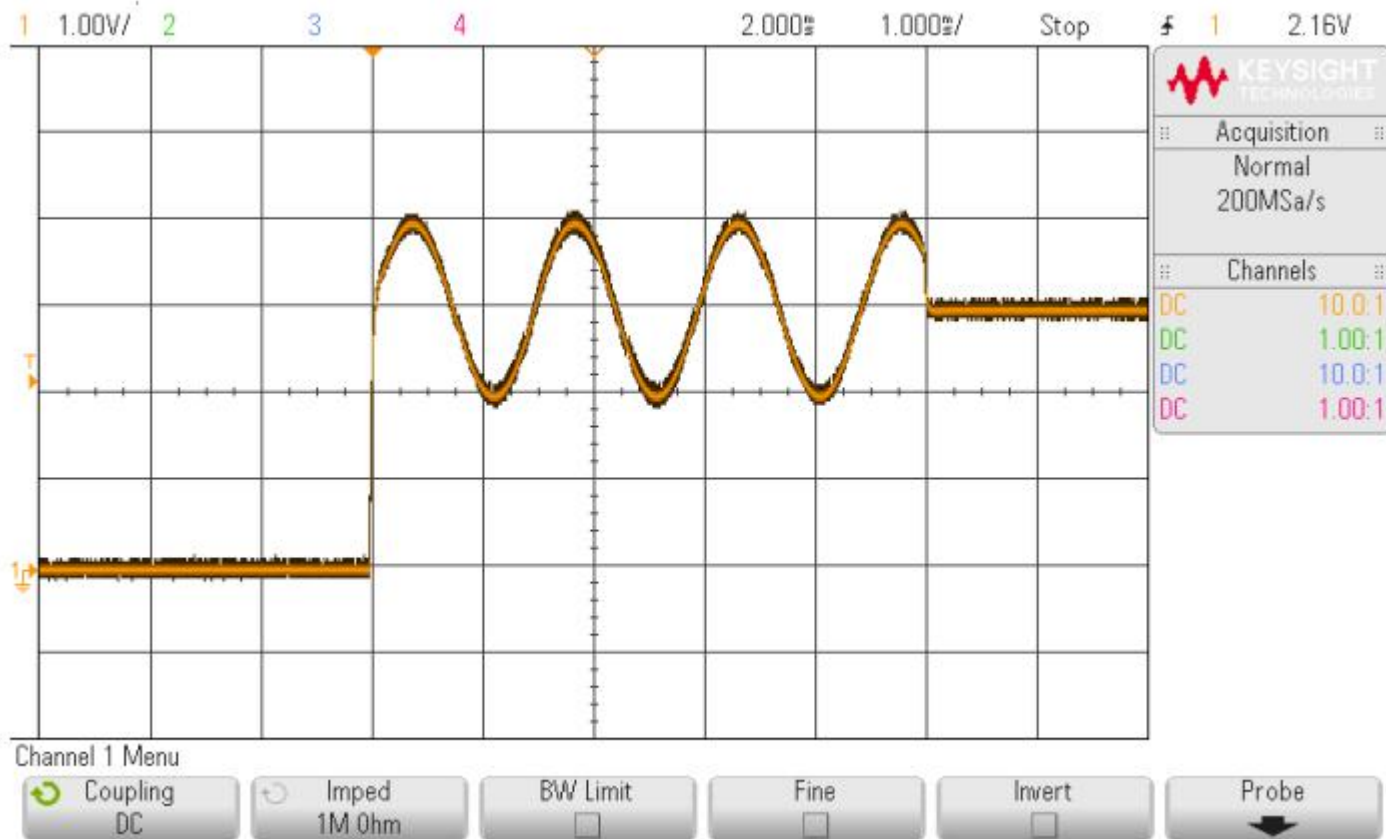
STSAWGCreateSineData(&awg_pattern[0], sam, 1, 2, 3, 0);
//Set a Sine wave data array, data size is 150, period is 1, Vpp=2V, DC offset = 3V, phase = 0.
fxvie0.AwgLoader("Sin_AWG", FV, FXVle_10V, FXVle_1A, awg_pattern, sam);
//Import awg_pattern data array into fxvie0 channel, pattern named as 'Sin_AWG'
fxvie0.Set(FV, 0, FXVle_10V, FXVle_1A, FXVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' ofits 'AwgLoader()'

fxvie0.AwgRun("Sin_AWG", 0, (sam - 1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
delay_ms(5); //fxvie0 AWG pattern keeps running for 5mS
fxvie0.AwgStop();//stop loops for fxvie0 AWG running
```

• FXVle –AWG

该正弦波周期为1.5mS，延时5mS后，停止awg 波形。

下图为示波器抓取的实际波形。



- FXVle –AWG

AwgClear(void);

Remarks

一般用于 AwgLoader 函数之前，清除该 FXVle 通道的 AWG 数据。

例1： Load 波形之前清除 fxvie0 通道的 AWG 数据。

```
fxvie0.AwgClear(); //clear AWG data of fxvie0  
fxvie0.AwgLoader("P1", FV, FXVle_30V, FXVle_100MA, awg_pattern1, sam);
```

• FXVIe –AWG

```
SetMeasVTrig(double vTrig, TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Remarks

电压测量触发设置。

注意:

SetMeasVTrig函数与 MeasureVI函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Parameters

vTrig double 型变量，设置触发电压。单位 V。

注意:

该参数的值与其对应通道所调用的 Set()函数的电压档位相关。

trigMode 触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

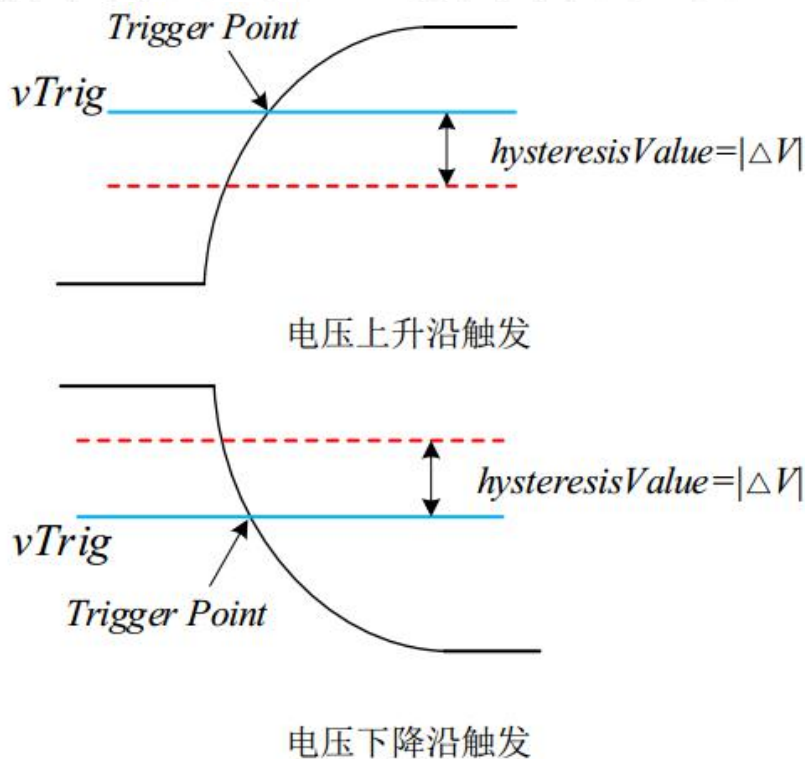
该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

hysteresisValue double 型变量，设置触发电压的滞回值。单位 V。在默认条件下等于 0。

• FXVIe –AWG

注意:

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值;
- 2) 在上升沿(TRIG_RISING)触发的情况下, 当判定到测量结果先小于($vTrig - hysteresisValue$) 值之后又大于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发;
- 3) 在下降沿(TRIG_FALLING)触发的情况下, 当判定到测量结果先大于($vTrig + hysteresisValue$)值之后又小于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发。



• FXVle –AWG

例1：建立一个波形数据组，fxvie0 从10.5V 下降到8V，步距0.1V。设置fxvie1下降沿触发，触发电平2V。

```

fxvie1.Set(FI, 0, FXVle_10V, FXVle_10MA, FXVle_RELAY_ON);
double Trig = 2; //triggering voltage
double Trig_Point[4] = { 0.0 };
double result[4] = { 0.0 };
int sam = 25; //data size of AWG pattern
double interval = 200; //interval time of AWG pattern, unit is uS
double awg_pattern[100] = { 0.0 };
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 10.5, 8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.
fxvie0.AwgLoader("Uvlo", FV, FXVle_30V, FXVle_100MA, awg_pattern, sam);
//import the awg_pattern array into fxvie0 channel, named it as 'Uvlo'. Set to FV mode.
fxvie0.Set(FV, 10.5, FXVle_30V, FXVle_100MA, FXVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'
fxvie0.AwgSelect("Uvlo", 0, sam - 1, sam - 1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
fxvie1.Set(FI, 0, FXVle_30V, FXVle_10MA, FXVle_RELAY_ON);
fxvie1.SetMeasVTrig(Trig, TRIG_FALLING); //trigger value is 2V, falling edge
fxvie0.MeasureVI(sam, interval, FXVle_MI_X1, MEAS_AWG);
fxvie1.MeasureVI(sam, interval, FXVle_MI_X1, MEAS_AWG);
STSEnableAWG(&fxvie0); //enable AWG pattern for fxvie0
STSEnableMeas(&fxvie0, &fxvie1); //enable measurement for fxvie0 and fxvie1
STSAWGRun(); //Enable AWG and measurement synchronously
    
```

SetMeasVTrig/SetMeasITrig函数与MeasureVI函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

• FXVIe –AWG

```
SetMeasITrig(double iTrig, TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Remarks

电流测量触发设置。

注意:

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Parameters

iTrig double 型变量，设置触发电流。单位 A。

注意:

该参数的值与其对应通道所调用的 Set()函数的电流档位相关。

trigMode 触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

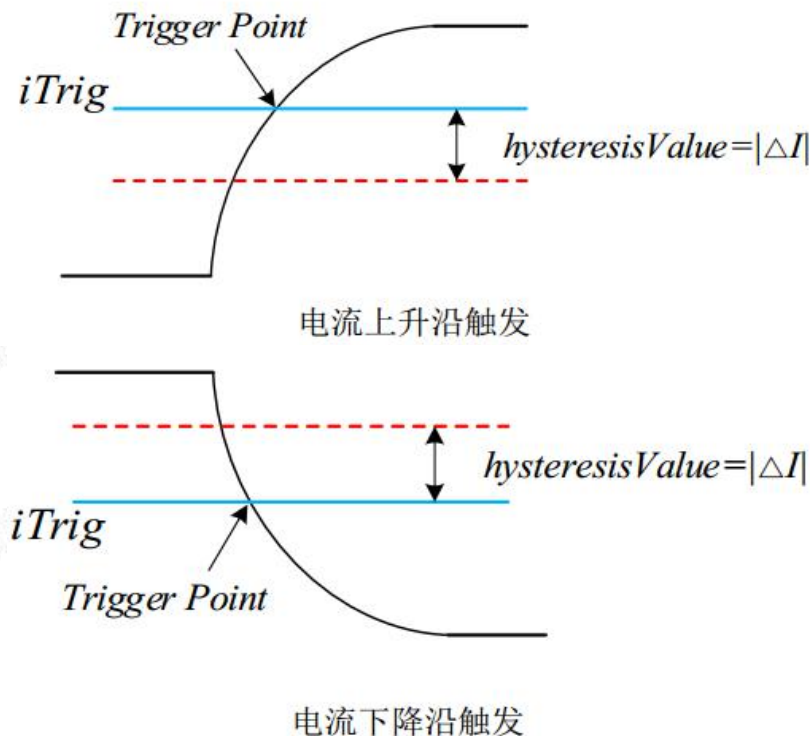
该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

hysteresisValue double 型变量，设置触发电流的滞回值。单位 A。在默认条件下等于 0。

• FXVIe –AWG

注意:

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值;
- 2) 在上升沿(TRIG_RISING)触发的情况下, 当判定到测量结果先小于($iTrig - hysteresisValue$) 值之后又大于*iTrig* 值时, 此时在*iTrig* 处判定为触发;
- 3) 在下降沿(TRIG_FALLING)触发的情况下, 当判定到测量结果先大于($iTrig + hysteresisValue$)值之后又小于*iTrig* 值时, 此时在*iTrig* 处判定为触发。



• FXVle –AWG&Digitizer 样例

```
DUT_API int test_VI_Example3(short funcindex, LPCTSTR funclabel)
```

```
{
```

```
//{{AFX_STG_PARAM_PROTOTYPES
```

```
CParam *Vcspre = StsGetParam(funcindex,"Vcspre");
```

```
//}}AFX_STG_PARAM_PROTOTYPES
```

```
// TODO: Add your function code here
```

```
int i = 0;
```

```
double Trig = 2;           // trigger level
```

```
double Trig_Point[STG_SITE_NUM] = { 0 };
```

```
double result[STG_SITE_NUM] = { 0 };
```

```
int sam = 100;              // awgSize
```

```
int interval = 100;         // awgInterval unit=us
```

```
double awg_pattern[100] = { 0.0 };
```

```
fxvie1.Set(FI, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_ON); //OUT Measure
```

```
delay_ms(1);
```

```
// Rampwave, awgData=0, awgSize=100, wavecycle=1, startValue=0.35V, stopValue=0.45V
```

```
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 0.35, 0.45);
```

```
// load the data of "awg_pattern"(sam=100), named "Vcspre_AWG"
```

```
fxvie0.AwgLoader("Vcspre_AWG", FV, FXVle_3p6V, FXVle_100MA, awg_pattern, sam);
```

```
// the V_range(FXVle_3p6V) and I_range(FXVle_100MA) should be the same with AwgLoader
```

```
fxvie0.Set(FV, 0.35, FXVle_3p6V, FXVle_100MA, FXVle_RELAY_ON);
```

```
// select AWGpattern of "Vcspre_AWG", startAddr=0, stopAddr=sam-1, loopBackAddr=sam-1, awgInterval=100us
```

```
fxvie0.AwgSelect("Vcspre_AWG", 0, sam - 1, sam - 1, interval);
```

• FXVle –AWG&Digitizer 样例

```
fxvie1.SetMeasVTrig(Trig, TRIG_FALLING);  
//set V_trigger level to 2V, Falling edge  
fxvie0.MeasureVI(500, 20, FXVle_MI_X1, MEAS_AWG);  
//fxvie0 AWG Mode, SampleTimes=500, samplePeriod=20us, MI X1.  
fxvie1.MeasureVI(500, 20, FXVle_MI_X1, MEAS_AWG);  
//fxvie1 AWG Mode, SampleTimes=500, samplePeriod=20us, MI X1.  
STSEnableAWG(&fxvie0);  
// fxvie0 enable AWG  
STSEnableMeas(&fxvie0, &fxvie1);  
// fxvie0 and fxvie1 enable AWG measure synchronously  
STSAWGRun();  
// AWG synchronous starting  
for (i = 0; i < STS_SITE_NUM; i++)  
{  
    Trig_Point[i] = fxvie1.GetMeasResult(i, MVRET, TRIG_RESULT);  
    // read the trigger point of fxvie1  
    result[i] = fxvie0.GetMeasResult(i, MVRET, Trig_Point[i]);  
    // fxvie0 read the V_value of trigger point  
    Vcspre->SetTestResult(i, 0, result[i]);  
}  
  
fxvie0.Set(FV, 0, FXVle_3p6V, FXVle_100MA, FXVle_RELAY_OFF);  
fxvie1.Set(FV, 0, FXVle_10V, FXVle_1MA, FXVle_RELAY_OFF);  
  
return 0;  
}
```

- FXVIe –TMU

TMUConnect ()

TMUDisconnect ()

TMUSetInSource ()

TMUSetMatrix ()

TMUStart ()

TMUStop ()

TMUMeasure ()

TMUGetMeasResult ()

TMUChannel ()

TMUChannelReset ()

- FXVle –TMU

TMUConnect (UINT*usDelay* = 500) ;

Remarks

接通当前 TMU 通道的输入继电器。

Parameters

usDelay UINT 型变量，设置接通继电器后的延时时间，单位uS，取值范围为0~10000，默认值为500。

```
FXVle fxvie4("S5_4, S5_10");
```

```
FXVle fxvie5("S5_5, S5_11");
```

```
fxvie4.TMUConnect();
```

- FXVle –TMU

```
TMUDisconnect (UINT usDelay = 100);
```

Remarks

断开当前逻辑通道的输入继电器。

Parameters

usDelay UINT 型变量，设置断开继电器后的延时时间，单位uS，取值范围为0~10000，默认值为 100。

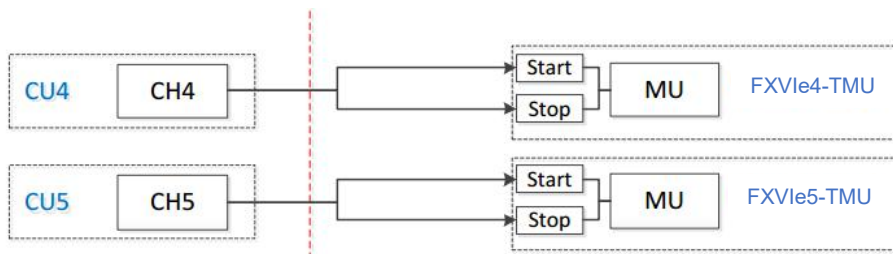
```
fxvie4.TMUDisconnect();
```

• FXVle –TMU

TMUSetInSource ();

Remarks

设置当前测量信号源为单信号源模式。 用户端通道单元和板卡内部测量单元一一对应。
典型单信号测量矩阵关系如下：



注意:

- 1) 该函数必须在TMUStart 和TMUStop 函数之前设置，保证实际设置到硬件的通道和TMUStart（TMUStop）函数中参数相关的通道保持一致；
- 2) 由 1）延伸，如果TMUSetInSource 在Start（Stop）函数之后设置，可能会导致两者使用的信号通道不匹配，在Measure 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值；
- 3) 系统每次测试结束后（InitAfterTestFlow 函数）后调用 TMUSetInSource，恢复默认单信号状态。
- 4) 如果需要设置复杂的矩阵关系，请参考函数 TMUSetMatrix。

```
fxvie4.TMUSetInSource(); // Singlesource
```

• FXVle –TMU

TMUSetMatrix (FXVle* *startChSel*, FXVle* *stopChSel*);

Remarks

设置用户端通道单元和板卡内部测量单元的矩阵关系，矩阵关系请参考下一页Example。

注意:

- 1) 该函数和 TMUSetInSource 都是设置矩阵，实际硬件以最后一次设置为准；
- 2) 该函数设置矩阵后，一直有效，除非设置 TMUSetInSource；
- 3) 该函数必须在 TMUStart 和 TMUStop 函数之前设置，保证实际设置到硬件的通道和 TMUStart (TMUStop) 函数中参数相关的通道保持一致；
- 4) 由 3) 延伸，如果 TMUSetMatrix 在 TMUStart (TMUStop) 函数之后设置，可能会导致两者的使用的信号通道不匹配，在 Measure 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值；
- 5) 该函数主要用于设置 TMU 为双信号测量模式，当设置单信号测量时，和 TMUSetInSource 函数功能一致；
- 6) 对于双信号测试，同一Bank 的两个TMU 通道必须在同一个Site；
- 7) 如果不满足以下情况，则不符合矩阵设置条件，不进行矩阵设置：
 - a) *startChSel* 和 *stopChSel* 包含的通道个数必须相同；
 - b) *startChSel* 和 *stopChSel* 一一对应的通道定义为一组信号通道，同一组信号通道必须在同一个bank 内，且为每个bank 的TMU 通道；
 - c) 当前定义的逻辑通道包含的物理通道，必须和 *startChSel* 或*stopChSel* 包含的物理通道相同；
 - d) 该函数设置矩阵，要么是双信号设置，要么是单信号设置，不能单信号和双信号混合设置。

• FXVle –TMU

Parameters

StartChSel FXVle 类型变量，设置 Start 信号源通道。

StopChSel FXVle 类型变量，设置 Stop 信号源通道。

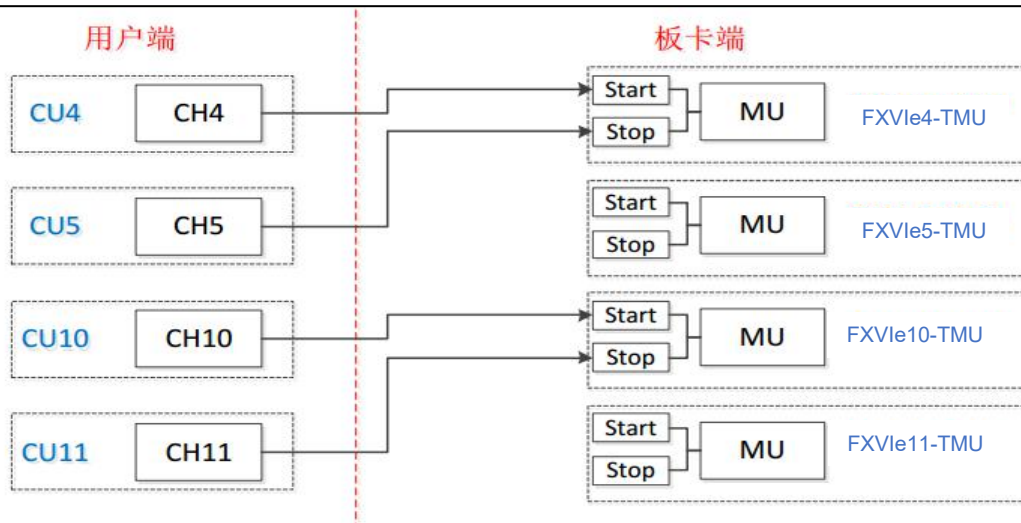
双工位配置，SITE_1 包含通道 4 和 5，SITE_2 包含通道 10 和 11，fxvie4 包含通道 4 和 10，fxvie5 包含通道 5 和 11。

例1：双信号测试，选择 fxvie4 的 MU 作为测量单元，fxvie4 包含的通道作为 start 信号，fxvie5 包含的通道作为 stop 信号。

```
FXVle fxvie4("S5_4, S5_10");
```

```
FXVle fxvie5("S5_5, S5_11");
```

```
fxvie4.TMUSetMatrix(&fxvie4, &fxvie5);
```



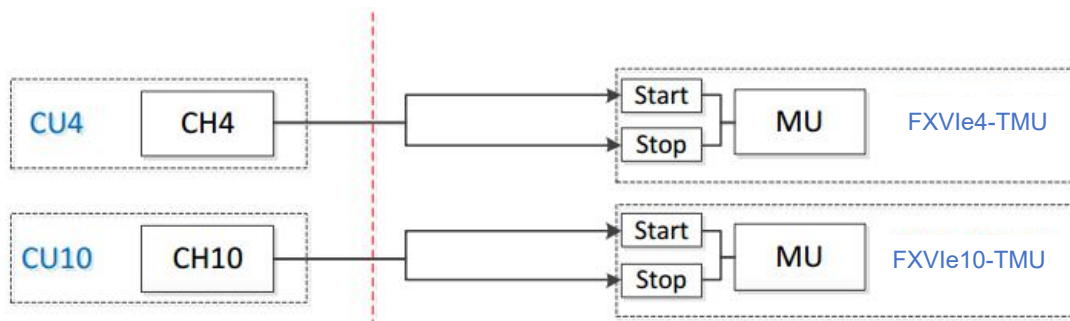
• FXVle –TMU

例2: 单信号测试, 选择 fxvie4 的 MU 作为测量单元, fxvie4 包含的通道分别作为start 和 stop 信号。

```
fxvie4.TMUSetMatrix(&fxvie4, &fxvie4);
```

上述 fxvie4 设置的矩阵和下面 TMUSetInSource 设置的矩阵相同

```
fxvie4.TMUSetInSource();
```



• FXVle –TMU

```
TMUStart(FXVle_VRNG vRange = FXVle_30V, FXVle_TMU_SLOPE slope  
= FXVle_TMU_POS, double triggerLevel = 0.0, FXVle_FILTER  
filter = FXVle_FILTER_PASS, UINT triggerWidth= 200,UINT  
holdoffTime= 0,UINT holdoffNum= 0);
```

Remarks

设置 START 信号。

注意:

当进行单信号测量时， TMUStart 和 TMUStop 函数的参数 VRange、 Filter 也必须设置一致。

Parameters

vRange 设置信号电压量程。

FXVle_30V: $\pm 30V$

FXVle_10V: $\pm 10V$

FXVle_3p6V: $\pm 3.6V$

默认值为 FXVle_30V。

Slope 设置比较器的触发极性。

FXVle_TMU_POS: 上升沿触发

FXVle_TMU_NEG: 下降沿触发

默认值为 FXVle_TMU_POS

triggerLevel 设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

• FXVle –TMU

Filter 设置滤波器。

FXVle_FILTER_PASS: 直通通路, 不使用滤波器

FXVle_FILTER_1MHz: 1MHz 低通滤波器

FXVle_FILTER_100KHz: 100KHz 低通滤波器

默认值为 FXVle_FILTER_PASS。

注意:

1)如果被测信号上有高频分量, 建议使用输入滤波器滤除该高频分量, 否则会影响测试结果;

2)被测信号为方波信号时不建议使用输入滤波器。

triggerWidth 设置触发脉宽, 取值范围 0~80000, 步进为 5, 单位 nS。

如果输入值不为 5 的整数倍, 将会内部取整为最接近输入值的 5 的整数倍。默认值为 200。

例如: 输入值为 18, 内部处理为 20。

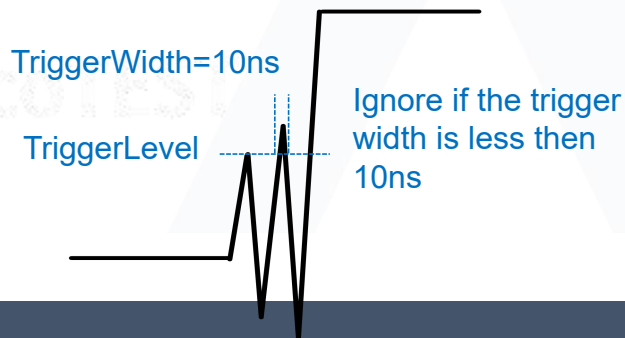
注意:

被测信号到达触发电平 **TriggerLevel** 之后, 若在 **TriggerWidth** 时间之内一直高于 (上升沿) 或低于 (下降沿) **TriggerLevel**, 则满足触发条件, 否则, 继续等待下次触发电平。

holdoffTime 设置延迟触发时间, 取值范围 0~1e9, 步进为 5, 单位 nS。如果输入值不为 5 的整数倍,

将会内部取整为最接近输入值的 5 的整数倍, 默认值为 0。例如: 输入值为 18, 内部处理为 20。

holdoffNum 忽略的 START 事件个数, 取值范围 0~65535, 默认值为 0。



• FXVle –TMU

例 1：设置 fxvie4 的 START 信号电压量程为 10V，上升沿 3.5V 触发，使用 1MHz滤波器。

```
fxvie4.TMUStart(FXVle_10V, FXVle_TMU_POS, 3.5, FXVle_FILTER_1MHz);
```

例 2：设置 fxvie4 的 START 信号电压量程为 3.6V，上升沿 1.5V 触发，不使用滤波器，触发脉宽为 10，延迟触发时间为 0，忽略 1 个 START 事件个数。

```
fxvie4.TMUStart(FXVle_3p6V, FXVle_TMU_POS, 1.5, FXVle_FILTER_PASS, 10, 0, 1);
```

• FXVle –TMU

```
TMUStop(FXVle_VRNG vRange= FXVle_30V,FXVle_TMU_SLOPE  
slope=FXVle_TMU_POS, double triggerLevel= 0.0, FXVle_FILTER  
filter= FXVle_FILTER_PASS, UINT triggerWidth= 200,  
UINT holdoffTime= 0,UINT holdoffNum= 0);
```

Remarks

设置 STOP信号。

注意:

- 1) 当进行单信号测量时，Start和 Stop函数的参数 VRange、Filter也必须设置一致。
- 2) 测量信号频率时，此函数可以不用调用。

Parameters

vRange 设置信号电压量程。

FXVle_30V: $\pm 30V$

FXVle_10V: $\pm 10V$

FXVle_3p6V: $\pm 3.6V$

默认值为 *FXVle_30V*。

Slope 设置比较器的触发极性。

FXVle_TMU_POS: 上升沿触发

FXVle_TMU_NEG: 下降沿触发

默认值为 *FXVle_TMU_POS*

triggerLevel 设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

• FXVle –TMU

Filter 设置滤波器。

FXVle_FILTER_PASS: 直通通路, 不使用滤波器

FXVle_FILTER_1MHz: 1MHz 低通滤波器

FXVle_FILTER_100KHz: 100KHz 低通滤波器

默认值为 FXVle_FILTER_PASS。

注意:

- 1)如果被测信号上有高频分量, 建议使用输入滤波器滤除该高频分量, 否则会影响测试结果;
- 2)被测信号为方波信号时不建议使用输入滤波器。

triggerWidth 设置触发脉宽, 取值范围 0~80000, 步进为 5, 单位 nS。

如果输入值不为 5 的整数倍, 将会内部取整为最接近输入值的 5 的整数倍。默认值为 200。

例如: 输入值为 18, 内部处理为 20。

注意:

被测信号到达触发电平 **TriggerLevel** 之后, 若在 **TriggerWidth** 时间之内一直高于 (上升沿) 或低于 (下降沿) **TriggerLevel**, 则满足触发条件, 否则, 继续等待下次触发电平。

holdoffTime 设置延迟触发时间, 取值范围 0~1e9, 步进为 5, 单位 nS。如果输入值不为 5 的整数倍, 将会内部取整为最接近输入值的 5 的整数倍, 默认值为 0。

例如: 输入值为 18, 内部处理为 20。

holdoffNum 忽略的 START 事件个数, 取值范围 0~65535, 默认值为 0。

• FXVle –TMU

例 1：设置 fxvie4 的 STOP 信号电压量程为 10V，下降沿 3.5V 触发，使用 1MHz滤波器。

```
fxvie4.TMUStop(FXVle_10V, FXVle_TMU_NEG, 3.5, FXVle_FILTER_1MHz);
```

例 2：设置 fxvie4 的 STOP 信号电压量程为 3.6V，上升沿 1.5V 触发，不使用滤波器，触发脉宽为 10，延迟触发时间为 0，忽略 2个 STOP 事件个数。

```
fxvie4.TMUStop(FXVle_3p6V, FXVle_TMU_POS, 1.5, FXVle_FILTER_PASS, 10, 0, 2);
```

• FXVle –TMU

```
TMUMeasure(FXVle_TMU_MEAS $measType$ = FXVle_MEAS_TIME,  
UINT $sampleNum$ = 1,double $msTimeout$ = 1.0,FXVle_TRNG $tRange$ =  
FXVle_TRANGE_MS);
```

Remarks

启动测量。

注意:

如果 TMUStart 和 TMUStop 函数中使用的信号通道和 TMUSetMatrix（或TMUSetInSource）函数中设置的信号通道不一致，则不进行测量，返回异常值。

Parameters

$measType$ 设置测量类型。

$FXVle_MEAS_TIME$: 测量时间

$FXVle_MEAS_FREQ$: 测量频率

$FXVle_MEAS_HIGH_DUTY$: 测量高电平在一个周期之内所占的时间比率

$FXVle_MEAS_LOW_DUTY$: 测量低电平在一个周期之内所占的时间比率

$FXVle_MEAS_EVENT$: 测量事件

默认值为 $FXVle_MEAS_TIME$ 。

$sampleNum$ 设置采样个数，取值范围为 1~2048，默认值为 1。

当 $MeasType$ 为 $FXVle_MEAS_TIME$ 类型时， $SampleNum$ 为采样次数；

当 $MeasType$ 为 $FXVle_MEAS_FREQ$ 类型时， $SampleNum$ 为周期数；

当 $MeasType$ 为 $FXVle_MEAS_HIGH_DUTY$ 或 $FXVle_MEAS_LOW_DUTY$ 类型时， $SampleNum$ 为跳变沿个数；

当 $MeasType$ 为 $FXVle_MEAS_EVENT$ 类型时， $SampleNum$ 为事件个数（包含 Start 和 Stop）。

• FXVle –TMU

注意:

当 MeasType 为 FXVle_MEAS_HIGH_DUTY 或 FXVle_MEAS_LOW_DUTY 类型时, 为保证正确测试, 要求 SampleNum 不小于 3 以采样至少一个周期。并且, Start()和 Stop()的 slope 设置必须一个为上升沿, 另一个为下降沿, 函数 Start()和 Stop()中的参数 holdoffNum 必须设置为 0。

msTimeout 设置测量等待时间, 等待 FXVle 完成设定次数采样, 取值范围 0.001~2.1e4。
单位mS, 默认值为 1mS。

注意:

如果超范围, 默认设置为 1。

tRange 设置测量时间量程。

FXVle_TRANGE_MS: ms 级量程档(30nS~5.368S)

FXVle_TRANGE_US: us 级量程档(800nS~4mS)

FXVle_TRANGE_NS: ns 级量程档(30nS~950nS)

默认值为 FXVle_TRANGE_MS。

注意:

MeasType 为 FXVle_MEAS_HIGH_DUTY、FXVle_MEAS_LOW_DUTY 或类型FXVle_MEAS_EVENT 时, Trange 必须为 FXVle_TRANGE_MS。

例: 设置 fxvie4 测量时间, 采样点数为 1, 超时时间为 2mS。

```
fxvie4.TMUMeasure(FXVle_MEAS_TIME, 1, 2);
```

• FXVle –TMU

TMUGetMeasResult (UINT *siteCount*, int *retType* = AVERAGE_RESULT);

Remarks

根据 TMUMeasure()函数中的 MeasType 参数类型，获取相应的测量结果。

Parameters

siteCount UINT 型变量，选择测量数据工位。 0 代表 Site1， 1 代表 Site2.....

retType 返回结果类型。

AVERAGE_RESULT

非负值 N ， $0 \leq N \leq \text{SampleNum}-1$

默认值为 AVERAGE_RESULT。

当 Measure()函数中的 MeasType 为 FXVle_MEAS_TIME 类型时， retType 为AVERAGE_RESULT 时，返回时间平均值； retType 为非负值 N 时，返回第 $N+1$ 次采样的测量值。单位为 uS。

当 Measure()函数中的 MeasType 为 FXVle_MEAS_FREQ 类型时， retType 参数无效，返回频率值，单位 KHz。

当 Measure()函数中的 MeasType 为 FXVle_MEAS_HIGH_DUTY 或FXVle_MEAS_LOW_DUTY 类型时， retType 参数无效，返回高低占空比，单位%。

当 Measure()函数中的 MeasType 为 FXVle_MEAS_EVENT 类型时， retType 为AVERAGE_RESULT 时，返回满足触发条件的事件个数； retType 为非负值 N 时，返回第 $N+1$ 次事件的时刻，单位为 uS。

• FXVle –TMU

例 1： fxvie4 获取工位 1 的时间测量结果。

```
fxvie4.TMUSetInSource();  
fxvie4.TMUStart(FXVle_30V, FXVle_TMU_POS, 1.5, FXVle_FILTER_PASS);  
fxvie4.TMUStop(FXVle_30V, FXVle_TMU_POS, 13.5, FXVle_FILTER_PASS);  
fxvie4.TMUConnect();  
fxvie4.TMUMeasure(FXVle_MEAS_TIME, 10, 20, FXVle_TRANGE_MS);  
//sampleNum=10, Timeout=20mS  
  
fxvie4.TMUGetMeasResult(0, AVERAGE_RESULT); //Get average time of site1  
fxvie4.TMUGetMeasResult(0, 5); // Get the of sixth point time of site1
```

例 2： fxvie4 获取工位 0 的高占空比测量结果。

```
fxvie4.TMUSetInSource();  
fxvie4.TMUStart(FXVle_10V, FXVle_TMU_POS, 2.0, FXVle_FILTER_PASS);  
fxvie4.TMUStop(FXVle_10V, FXVle_TMU_NEG, 2.0, FXVle_FILTER_PASS);  
fxvie4.TMUConnect();  
fxvie4.TMUMeasure(FXVle_MEAS_HIGH_DUTY, 20, 10, FXVle_TRANGE_MS);  
//sampleNum=20, Timeout=10mS  
  
fxvie4.TMUGetMeasResult(0, AVERAGE_RESULT); //Get average time of site1
```

• FXVle –TMU

```
DUT_API int test_TMU_Freq(short funcindex, LPCTSTR funclabel)
{
    //{AFX_STG_PARAM_PROTOTYPES
    CParam *Freq = StsGetParam(funcindex, "Freq");
    //}}AFX_STG_PARAM_PROTOTYPES
    // TODO: Add your function code here
    double val[4] = { 0.0 };
    int i = 0;

    fxvie4.TMUConnect();
    fxvie4.TMUSetInSource();
    fxvie4.TMUStart(FXVle_10V, FXVle_TMU_POS, 2.0, FXVle_FILTER_PASS, 10);
    fxvie4.TMUMeasure(FXVle_MEAS_FREQ, 1, 10, FXVle_TRANGE_US);

    for (i = 0; i < STS_SITE_NUM; i++)
    {
        val[i] = fxvie4.TMUGetMeasResult(i, AVERAGE_RESULT);
        Freq->SetTestResult(i, 4, val[i]); //KHz
    }

    fxvie4.TMUDisconnect();

    return 0;
}
```

• FXVie –TMU

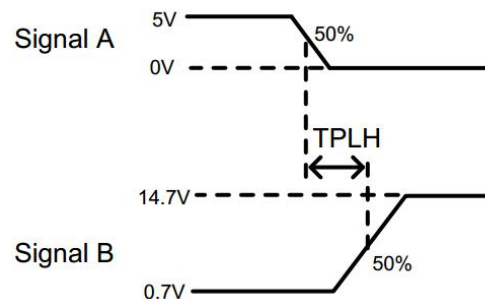
```

DUT_API int test_TMU_TPLH(short funcindex, LPCTSTR funclabel)
{
    //{AFX_STS_PARAM_PROTOTYPES
    CParam *TPLH = StsGetParam(funcindex, "TPLH");
    //}}AFX_STS_PARAM_PROTOTYPES
    // TODO: Add your function code here
    double val[4] = { 0.0 };
    int i = 0;

    fxvie4.TMUConnect();
    fxvie4.TMUSetMatrix(&fxvie4, &fxvie5);
    //SH4_FANOUT connect signal A, SH5_FANOUT connect signal B

    fxvie4.TMUStart(FXVie_30V, FXVie_TMU_NEG, 2.5, FXVie_FILTER_PASS);
    fxvie4.TMUStop(FXVie_30V, FXVie_TMU_POS, 7.7, FXVie_FILTER_PASS);
    fxvie4.TMUMeasure(FXVie_MEAS_TIME, 1, 10, FXVie_TRANGE_US);
    for (i = 0; i < STS_SITE_NUM; i++)
    {
        val[i] = fxvie4.TMUGetMeasResult(i, AVERAGE_RESULT);
        TPLH->SetTestResult(i, 0, val[i]); //us
    }
    fxvie4.TMUDisconnect();

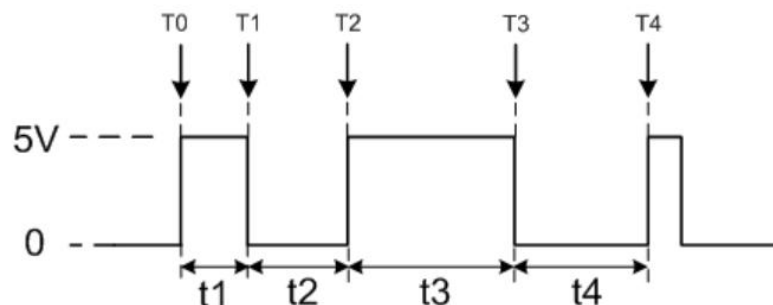
    return 0;
}
    
```



• FXVie –TMU

```

DUT_API int test_TMU_Event(short funcindex, LPCTSTR funclabel)
{
    //{{AFX_STG_PARAM_PROTOTYPES
    CParam *t1 = StsGetParam(funcindex, "t1");
    CParam *t2 = StsGetParam(funcindex, "t2");
    CParam *t3 = StsGetParam(funcindex, "t3");
    CParam *t4 = StsGetParam(funcindex, "t4");
    //}}AFX_STG_PARAM_PROTOTYPES
    // TODO: Add your function code here
    double Tevent[5][STG_SITE_NUM] = { 0.0 };
    fxvie4.TMUConnect();
    fxvie4.TMUSetInSource();
    fxvie4.TMUStart(FXVie_10V, FXVie_TMU_POS, 2.5, FXVie_FILTER_PASS, 10);
    fxvie4.TMUStop(FXVie_10V, FXVie_TMU_NEG, 2.4, FXVie_FILTER_PASS, 10);
    fxvie4.TMUMeasure(FXVie_MEAS_EVENT, 5, 10, FXVie_TRANGE_MS);
    //Apply excitation to generate signal
    for (int SITEID = 0; SITEID < STG_SITE_NUM; SITEID ++)
    {
        for (int i = 0; i < 5; i++){
            Tevent[i][SITEID] = fxvie4.TMUGetMeasResult(SITEID, i);
        }
        t1->SetTestResult(SITEID, 0, (Tevent[1][SITEID] - Tevent[0][SITEID]));
        t2->SetTestResult(SITEID, 0, (Tevent[2][SITEID] - Tevent[1][SITEID]));
        t3->SetTestResult(SITEID, 0, (Tevent[3][SITEID] - Tevent[2][SITEID]));
        t4->SetTestResult(SITEID, 0, (Tevent[4][SITEID] - Tevent[3][SITEID]));
    }
    fxvie4.TMUDisconnect();
    return 0;
}
    
```



• FXVIe –TMU

TMUChannel (FXVIe *FXVIeChannel*) ;

Remarks

将 bank 内的通道通过内部总线连接到 TMU 通道， 让该通道具有时间测量功能。

注意:

- 1) 该函数不支持 group;
- 2) TMU 通道及其搭接通道属性必须都是多工位或都是 NOSITE;
- 3) 相同工位下， TMU 通道及其搭接通道必须在同一 bank;
- 4) 不同工位下， TMU 通道及其搭接通道必须在不同 bank;
- 5) 对于相同 bank 的任一通道，使用该函数后，若未进行复位操作，则该 bank 所有通道的 GangRelay 函数设置无效;
- 6) 对于 TMU 通道及其搭接通道，使用该函数后，若未进行复位操作，则 Set 函数设置无效;
- 7) 每个 bank 仅有一个内部总线，同一时刻仅支持 bank 内一个通道连接到 TMU 通道。

Parameters

FXVIeChannel FXVIe 类型变量， 设置需要 TMU 功能的通道。

• FXVIe –TMU

例 1： 将 fxvie0 搭接到 TMU 通道 fxvie4 上

```
FXVIe fxvie0("S18_0,S18_6", "fxvie0");
FXVIe fxvie4("S18_4,S18_10", "fxvie4");
STSSetMultiSiteBind(MD_FXVIe,SITE_1,"S18_0,S18_1,S18_2,S18_3,S18_4,S18_5");
STSSetMultiSiteBind(MD_FXVIe,SITE_2,"S18_6,S18_7,S18_8,S18_9,S18_10,S18_11");

fxvie4.TMUChannel(fxvie0);
fxvie4.TMUConnect();
fxvie4.TMUSetInSource();
fxvie4.TMUStart(FXVIe_10V, FXVIe_TMU_POS, 3, FXVIe_FILTER_PASS, 100);
fxvie4.TMUStop(FXVIe_10V, FXVIe_TMU_NEG, 3, FXVIe_FILTER_PASS, 100);
fxvie4.TMUMeasure(FXVIe_MEAS_TIME, 1, 1.0, FXVIe_TRANGE_MS);
for (int i = 0; i < STS_SITE_NUM; i++)
{
    Time_Result[i] = fxvie4.TMUGetMeasResult(i);
    Gang_TMU->SetTestResult(i, 0, Time_Result[i]);
}
fxvie4.TMUDisconnect();
fxvie4.TMUChannelReset();
```

- FXVle –TMU

TMUChannelReset();

Remarks

TMU 通道搭接功能复位。

继电器恢复 RelayOff。

注意:

在进行其它非 TMU 操作之前，必须先使用该函数解除 TMU 通道搭接。

例 1： 将 fxvie0 和 fxvie4 的 TMU 通道断开搭接。

```
fxvie4.TMUChannel(fxvie0);  
fxvie4.TMUChannelReset();
```

- FXVle –DVM

FXVleDiffMeasure()

FXVleDiffGetMeasResult()

FXVleDiffBlockRead()

• FXVle –DVM

```
FXVleDiffMeasure (FXVle*baseFXVle, FXVle*measFXVle,  
UINTsampleTimes, double samplePeriod,  
FXVle_DIFF_FILTER filter = FXVle_DIFF_FILTER_OFF);
```

Remarks

设置差分测量通道，并启动差分测量。

注意：

- 1) 该函数为全局函数；
- 2) 进行差分的两个通道必须在同一个 bank 内；
- 3) 不支持两个相同的通道进行差分测量；
- 4) 该函数不支持 GROUP 操作；
- 5) *baseFXVle* 和 *measFXVle* 包含的物理通道数必须相同。

Parameters

baseFXVle FXVle 类型变量，基准通道。

measFXVle FXVle 类型变量，被测通道。

sampleTimes UINT 类型变量，差分测量采样点数，范围 1~2048。

samplePeriod double 类型变量，差分测量采样间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。
范围：5~30000。

Filter 设置差分测量滤波器状态。

FXVle_DIFF_FILTER_OFF: 关闭滤波器

FXVle_DIFF_FILTER_ON: 接通滤波器

默认值为 *FXVle_DIFF_FILTER_OFF*。

- FXVle –DVM

双工位配置，SITE_1包含通道0和1，SITE_2包含通道6和7，fxvie0 包含通道0 和6，fxvie1包含通道1和 7。

```
FXVle fxvie0("S5_0, S5_6");  
FXVle fxvie1("S5_1, S5_7");
```

例1：测试通道 0 和 1、通道 6 和 7 之间的差分结果，采样测试为 100，采样间隔10uS。

```
FXVleDiffMeasure(&fxvie0, &fxvie1, 100, 10);
```

• FXVle –DVM

```
FXVleDiffGetMeasResult (UINT siteCount,  
int sampleNumber = AVERAGE_RESULT);
```

Remarks

获取 FXVleDiffMeasure()函数的测量结果，单位为 V。

注意:

该函数为全局函数；测量完成后需及时回读测量结果，否则下次差分测量时，会覆盖上次测量结果。

Parameters

siteCount UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

非负值 N: 返回的是第 N+1 个采样点的测量值。假设 FXVleDiffMeasure的采样次数为 sam，
则 $0 \leq N \leq \text{sam}-1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

例 1: 返回 fxvie0 和 fxvie1 的差分测量结果。

```
FXVleDiffMeasure(&fxvie0, &fxvie1, 100, 10);  
for (i = 0; i < SITENUM; i++)  
{  
    adresult[i] = FXVleDiffGetMeasResult(i);  
}
```

• FXVle –DVM

```
FXVleDiffBlockRead(UINT siteCount,UINT start,UINT size,  
double* buffer);
```

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

注意：

该函数为全局函数；

Parameters

siteCount UINT 型变量，选择测量数据工位。 0 代表 Site1， 1 代表 Site2.....

start UINT 型变量，数据块的起始地址，非负整数。

size UINT 型变量，数据块的大小，正整数。

buffer 存放数据结果的数组，由用户定义，如： `double result[1000]={0.0};`

注意：

- 1) *buffer* 数组长度需不小于 *size* 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

```
double result[SITENUM][200] = { 0.0 };  
FXVleDiffMeasure(&fxvie0, &fxvie1, 100, 10);  
for (i = 0; i < SITENUM; i++)  
{  
    // start=0; size=200;  
    FXVleDiffBlockRead(i, 0, 200, result[i]);  
}
```

• FXVle –DVM

```
DUT_API int test_DVM_Example(short funcindex, LPCTSTR funclabel)
{
    //{AFX_STS_PARAM_PROTOTYPES
    CParam *Dvm = StsGetParam(funcindex, "Dvm");
    //}}AFX_STS_PARAM_PROTOTYPES
    // TODO: Add your function code here
    double val[4] = { 0.0 };
    int i = 0;

    fxvie1.Set(FI, 0.0, FXVle_10V, FXVle_100UA, FXVle_RELAY_ON);
    delay_ms(1);
    fxvie0.Set(FV, 1.2, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
    delay_ms(1);

    double result[SITENUM][200] = { 0.0 };
    FXVleDiffMeasure(&fxvie0, &fxvie1, 200, 10);
    for (i = 0; i < SITENUM; i++)
    {
        adresult[i] = FXVleDiffGetMeasResult(i, AVERAGE_RESULT); //Return Vfxvie1-Vfxvie0
        // start=0; size=200;
        FXVleDiffBlockRead(i, 0, 200, result[i]);
    }
    fxvie0.Set(FV, 0.0, FXVle_10V, FXVle_100MA, FXVle_RELAY_ON);
    delay_ms(1);
    fxvie1.Set(FI, 0.0, FXVle_10V, FXVle_100UA, FXVle_RELAY_ON);
    delay_ms(1);

    return 0;
}
```



Make Testing More Valuable